

Decentralized Optimization Algorithms: Motivation and Basics

Ming Yan

School of Data Science

The Chinese University of Hong Kong, Shenzhen



香港中文大學(深圳)

The Chinese University of Hong Kong, Shenzhen



SCHOOL OF
DATA SCIENCE
數據科學學院

Outline

Introduction

Average Consensus

Decentralized algorithms

DGD

EXTRA/PG-EXTRA

NIDS

General framework

Accelerated algorithms

OGT: Optimal Gradient Tracking with Loopless Chebyshev Acceleration

OPAPC: Optimized PAPC with Chebyshev Acceleration

Algorithms with Compression

LEAD: Linear convergent decentralized algorithm with compression

Directed Network

APD

Why Decentralized Optimization?

- No central coordinator or server
- Local data, global decision
- Applications:
 - UAV formation control
 - Smart grid state estimation
 - Federated learning

Problem Formulation

Consensus Optimization

$$\min_{x \in \mathbb{R}^d} \bar{f}(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Each agent i knows only $f_i(x)$ and communicates with neighbors in a graph.

Average Consensus

One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

Average Consensus

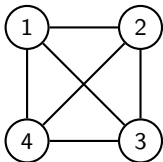
One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

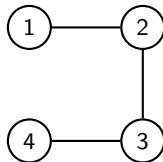
where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

- $\mathbf{W}$ is the **weight matrix** that encodes the network. Nonzero entries correspond to edges. Assume $\mathbf{W}$ is **symmetric** $\Rightarrow$ the network is **undirected**.

$$\mathbf{W} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



$$\mathbf{W} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix}$$



Average Consensus

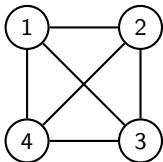
One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

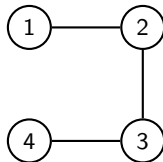
where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

- $\mathbf{W}$ is the **weight matrix** that encodes the network. Nonzero entries correspond to edges. Assume $\mathbf{W}$ is **symmetric** $\Rightarrow$ the network is **undirected**.

$$\mathbf{W} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



$$\mathbf{W} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix}$$



- $\mathbf{W}$ has an eigenvalue of 1 with eigenvector $\mathbf{1}_{n \times 1}$. All row (and column) sums are 1.

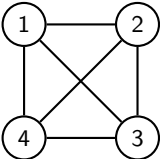
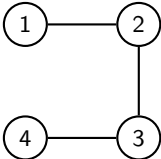
Average Consensus

One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

- $\mathbf{W}$ is the **weight matrix** that encodes the network. Nonzero entries correspond to edges. Assume $\mathbf{W}$ is **symmetric** $\Rightarrow$ the network is **undirected**.

$$\mathbf{W} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix}$$


- $\mathbf{W}$ has an eigenvalue of 1 with eigenvector $\mathbf{1}_{n \times 1}$. All row (and column) sums are 1.
- Any fixed point is a consensus vector: $\mathbf{x}^* = c\mathbf{1}_{n \times 1}$.

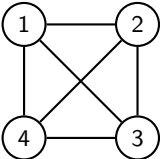
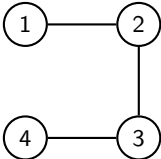
Average Consensus

One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

- $\mathbf{W}$ is the **weight matrix** that encodes the network. Nonzero entries correspond to edges. Assume $\mathbf{W}$ is **symmetric** $\Rightarrow$ the network is **undirected**.

$$\mathbf{W} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix}$$


- $\mathbf{W}$ has an eigenvalue of 1 with eigenvector $\mathbf{1}_{n \times 1}$. All row (and column) sums are 1.
- Any fixed point is a consensus vector: $\mathbf{x}^* = c\mathbf{1}_{n \times 1}$.
- All other eigenvalues of $\mathbf{W}$ lie in the interval $(-1, 1)$.

Average Consensus

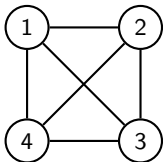
One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

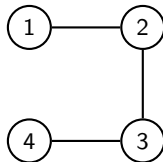
where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

- $\mathbf{W}$ is the **weight matrix** that encodes the network. Nonzero entries correspond to edges. Assume $\mathbf{W}$ is **symmetric** $\Rightarrow$ the network is **undirected**.

$$\mathbf{W} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



$$\mathbf{W} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix}$$



- $\mathbf{W}$ has an eigenvalue of 1 with eigenvector $\mathbf{1}_{n \times 1}$. All row (and column) sums are 1.
- Any fixed point is a consensus vector: $\mathbf{x}^* = c\mathbf{1}_{n \times 1}$.
- All other eigenvalues of $\mathbf{W}$ lie in the interval $(-1, 1)$.
- The total sum remains constant: $\mathbf{1}^\top \mathbf{x}^{k+1} = \mathbf{1}^\top \mathbf{W}\mathbf{x}^k = \mathbf{1}^\top \mathbf{x}^k$.

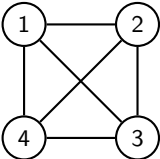
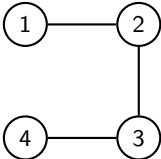
Average Consensus

One iteration:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k,$$

where $\mathbf{x} = [x_1, \dots, x_n]^\top$ and x_i is the value at agent i .

- $\mathbf{W}$ is the **weight matrix** that encodes the network. Nonzero entries correspond to edges. Assume $\mathbf{W}$ is **symmetric** $\Rightarrow$ the network is **undirected**.

$$\mathbf{W} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix}$$


- $\mathbf{W}$ has an eigenvalue of 1 with eigenvector $\mathbf{1}_{n \times 1}$. All row (and column) sums are 1.
- Any fixed point is a consensus vector: $\mathbf{x}^* = c\mathbf{1}_{n \times 1}$.
- All other eigenvalues of $\mathbf{W}$ lie in the interval $(-1, 1)$.
- The total sum remains constant: $\mathbf{1}^\top \mathbf{x}^{k+1} = \mathbf{1}^\top \mathbf{W}\mathbf{x}^k = \mathbf{1}^\top \mathbf{x}^k$.
- The convergence speed depends on the **second largest eigenvalue (in magnitude)** of $\mathbf{W}$.

Decentralized Averaging as Gradient Descent

Decentralized Averaging:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k$$

Decentralized Averaging as Gradient Descent

Decentralized Averaging:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k$$

We can rewrite the iteration as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{I} - \mathbf{W})\mathbf{x}^k$$

Decentralized Averaging as Gradient Descent

Decentralized Averaging:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k$$

We can rewrite the iteration as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{I} - \mathbf{W})\mathbf{x}^k$$

This is equivalent to applying **gradient descent with stepsize 1** to the following problem:

$$\min_{\mathbf{x}} \frac{1}{2} \left\| \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} \right\|_2^2$$

Decentralized Averaging as Gradient Descent

Decentralized Averaging:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k$$

We can rewrite the iteration as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{I} - \mathbf{W})\mathbf{x}^k$$

This is equivalent to applying **gradient descent with stepsize 1** to the following problem:

$$\min_{\mathbf{x}} \frac{1}{2} \left\| \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} \right\|_2^2$$

The solution converges to a consensus vector that preserves the initial average:

$$\mathbf{x}^* = c\mathbf{1} \quad \text{where } c = \frac{1}{n} \mathbf{1}^\top \mathbf{x}^0$$

Decentralized Averaging as Gradient Descent

Decentralized Averaging:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k$$

We can rewrite the iteration as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{I} - \mathbf{W})\mathbf{x}^k$$

This is equivalent to applying **gradient descent with stepsize 1** to the following problem:

$$\min_{\mathbf{x}} \frac{1}{2} \|\sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x}\|_2^2$$

The solution converges to a consensus vector that preserves the initial average:

$$\mathbf{x}^* = c\mathbf{1} \quad \text{where } c = \frac{1}{n}\mathbf{1}^\top \mathbf{x}^0$$

- The gradient is $(\mathbf{I} - \mathbf{W})\mathbf{x}$, which is **Lipschitz continuous** with constant < 2 .
- Therefore, a stepsize of 1 ensures convergence.

Outline

Introduction

Average Consensus

Decentralized algorithms

DGD

EXTRA/PG-EXTRA

NIDS

General framework

Accelerated algorithms

OGT: Optimal Gradient Tracking with Loopless Chebyshev Acceleration

OPAPC: Optimized PAPC with Chebyshev Acceleration

Algorithms with Compression

LEAD: Linear convergent decentralized algorithm with compression

Directed Network

APD

Decentralized Gradient Descent (DGD)

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i), \quad \text{subject to } x_1 = x_2 = \cdots = x_n.$$

Decentralized Gradient Descent (DGD)

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i), \quad \text{subject to } \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

The **Decentralized Gradient Descent (DGD)** algorithm (Nedić–Ozdaglar, 2009 TAC) updates:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k),$$

where $\mathbf{W}$ is the mixing matrix, and γ is the stepsize.

Decentralized Gradient Descent (DGD)

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i), \quad \text{subject to } \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

The **Decentralized Gradient Descent (DGD)** algorithm (Nedić–Ozdaglar, 2009 TAC) updates:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k),$$

where $\mathbf{W}$ is the mixing matrix, and γ is the stepsize.

- This can be rewritten as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \left((\mathbf{I} - \mathbf{W})\mathbf{x}^k + \gamma \nabla f(\mathbf{x}^k) \right).$$

Decentralized Gradient Descent (DGD)

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i), \quad \text{subject to } \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

The **Decentralized Gradient Descent (DGD)** algorithm (Nedić–Ozdaglar, 2009 TAC) updates:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k),$$

where $\mathbf{W}$ is the mixing matrix, and γ is the stepsize.

- This can be rewritten as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \left((\mathbf{I} - \mathbf{W})\mathbf{x}^k + \gamma \nabla f(\mathbf{x}^k) \right).$$

- Interpretation: DGD is gradient descent with step size 1 applied to:

$$\min_{\mathbf{x}} \frac{1}{2} \left\| \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} \right\|_2^2 + \gamma f(\mathbf{x}).$$

Decentralized Gradient Descent (DGD)

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i), \quad \text{subject to } \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

The **Decentralized Gradient Descent (DGD)** algorithm (Nedić–Ozdaglar, 2009 TAC) updates:

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k),$$

where $\mathbf{W}$ is the mixing matrix, and γ is the stepsize.

- This can be rewritten as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \left((\mathbf{I} - \mathbf{W})\mathbf{x}^k + \gamma \nabla f(\mathbf{x}^k) \right).$$

- Interpretation: DGD is gradient descent with step size 1 applied to:

$$\min_{\mathbf{x}} \frac{1}{2} \left\| \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} \right\|_2^2 + \gamma f(\mathbf{x}).$$

- The solution is generally **not a consensus**, since:

$$\mathbf{W}\mathbf{x}^* = \mathbf{x}^* + \gamma \nabla f(\mathbf{x}^*) \neq \mathbf{x}^*.$$

Decentralized Gradient Descent (DGD)

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i), \quad \text{subject to } \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

The **Decentralized Gradient Descent (DGD)** algorithm (Nedić–Ozdaglar, 2009 TAC) updates:

$$\mathbf{x}^{k+1} = \mathbf{W} \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k),$$

where $\mathbf{W}$ is the mixing matrix, and γ is the stepsize.

- This can be rewritten as:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - ((\mathbf{I} - \mathbf{W}) \mathbf{x}^k + \gamma \nabla f(\mathbf{x}^k)).$$

- Interpretation: DGD is gradient descent with step size 1 applied to:

$$\min_{\mathbf{x}} \frac{1}{2} \left\| \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} \right\|_2^2 + \gamma f(\mathbf{x}).$$

- The solution is generally **not a consensus**, since:

$$\mathbf{W} \mathbf{x}^* = \mathbf{x}^* + \gamma \nabla f(\mathbf{x}^*) \neq \mathbf{x}^*.$$

- A common fix is to use a **diminishing stepsize** $\gamma^k \rightarrow 0$ to guarantee convergence to consensus.

Decentralized Gradient Descent (DGD)

Update Rule

$$x_i^{k+1} = \sum_{j \in \mathcal{N}_i} W_{ij} x_j^k - \gamma \nabla f_i(x_i^k)$$

- Agents combine local averaging and gradient steps
- Fixed step size $\Rightarrow$ converges to a neighborhood
- Requires diminishing γ for convergence $\Rightarrow$ slower convergence

Overcoming DGD Limitations

Decentralized Gradient Descent (DGD) has several limitations, especially with constant step sizes and exact convergence. Several approaches have been proposed to address these issues:

Overcoming DGD Limitations

Decentralized Gradient Descent (DGD) has several limitations, especially with constant step sizes and exact convergence. Several approaches have been proposed to address these issues:

1. Exact & Constant-Step Methods

- **EXTRA/PG-EXTRA** — Exact first-order methods (Shi et al., 2015)
- **DIGing** — Gradient tracking with constant step size (Qu & Li, 2016; Nedić et al., 2017)
- **NIDS** — Network-independent step size (Li et al., 2019)

Overcoming DGD Limitations

Decentralized Gradient Descent (DGD) has several limitations, especially with constant step sizes and exact convergence. Several approaches have been proposed to address these issues:

1. Exact & Constant-Step Methods

- **EXTRA/PG-EXTRA** — Exact first-order methods (Shi et al., 2015)
- **DIGing** — Gradient tracking with constant step size (Qu & Li, 2016; Nedić et al., 2017)
- **NIDS** — Network-independent step size (Li et al., 2019)

2. Alternating Direction Method of Multipliers (ADMM)-Based Methods

- ADMM *(Shi et al., 2014; Chang et al., 2015; Hong & Chang, 2017)*

Overcoming DGD Limitations

Decentralized Gradient Descent (DGD) has several limitations, especially with constant step sizes and exact convergence. Several approaches have been proposed to address these issues:

1. Exact & Constant-Step Methods

- **EXTRA/PG-EXTRA** — Exact first-order methods (Shi et al., 2015)
- **DIGing** — Gradient tracking with constant step size (Qu & Li, 2016; Nedić et al., 2017)
- **NIDS** — Network-independent step size (Li et al., 2019)

2. Alternating Direction Method of Multipliers (ADMM)-Based Methods

- ADMM *(Shi et al., 2014; Chang et al., 2015; Hong & Chang, 2017)*

3. Multi-Consensus Approaches

- Inner-loop consensus to ensure accuracy *(Chen & Ozdaglar, 2012; Jakovetić et al., 2014)*

Overcoming DGD Limitations

Decentralized Gradient Descent (DGD) has several limitations, especially with constant step sizes and exact convergence. Several approaches have been proposed to address these issues:

1. Exact & Constant-Step Methods

- **EXTRA/PG-EXTRA** — Exact first-order methods (Shi et al., 2015)
- **DIGing** — Gradient tracking with constant step size (Qu & Li, 2016; Nedić et al., 2017)
- **NIDS** — Network-independent step size (Li et al., 2019)

2. Alternating Direction Method of Multipliers (ADMM)-Based Methods

- ADMM *(Shi et al., 2014; Chang et al., 2015; Hong & Chang, 2017)*

3. Multi-Consensus Approaches

- Inner-loop consensus to ensure accuracy *(Chen & Ozdaglar, 2012; Jakovetić et al., 2014)*

4. Acceleration Techniques

- Momentum-based acceleration *(Qu & Li, 2017, Song et al., 2024)*
- Dual acceleration and optimal rates *(Scaman et al., 2017; Uribe et al., 2017)*

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

- **Lagrangian:**

$$\mathcal{L}(\mathbf{x}, \mathbf{s}) = f(\mathbf{x}) + \langle \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}, \mathbf{s} \rangle,$$

where $\mathbf{s}$ is the Lagrange multiplier.

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

- **Lagrangian:**

$$\mathcal{L}(\mathbf{x}, \mathbf{s}) = f(\mathbf{x}) + \langle \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}, \mathbf{s} \rangle,$$

where $\mathbf{s}$ is the Lagrange multiplier.

- **KKT Conditions (Optimality):**

$$\mathbf{0} = \nabla f(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*,$$

$$\mathbf{0} = \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*.$$

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

- **Lagrangian:**

$$\mathcal{L}(\mathbf{x}, \mathbf{s}) = f(\mathbf{x}) + \langle \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}, \mathbf{s} \rangle,$$

where $\mathbf{s}$ is the Lagrange multiplier.

- **KKT Conditions (Optimality):**

$$\begin{aligned} \mathbf{0} &= \nabla f(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*, \\ \mathbf{0} &= \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*. \end{aligned}$$

- **Compact Matrix Form:**

$$-\begin{bmatrix} \nabla f(\mathbf{x}^*) \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^* \\ \mathbf{s}^* \end{bmatrix}.$$

Forward-Backward Splitting for KKT System

- KKT system of the decentralized problem:

$$= - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} + \begin{bmatrix} \mathbf{0} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}$$

Forward-Backward Splitting for KKT System

- Using forward-backward on the KKT system:

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ &= \begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} + \begin{bmatrix} \mathbf{0} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} \end{aligned}$$

- It simplifies to:

$$\begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ -2\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}$$

Forward-Backward Splitting for KKT System

- It simplifies to:

$$\begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ -2\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}$$

- Component-wise form:

$$\begin{aligned} \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^k &= \alpha \mathbf{x}^{k+1}, \\ -\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^k + \beta \mathbf{s}^k &= -2\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^{k+1} + \beta \mathbf{s}^{k+1} \end{aligned}$$

Forward-Backward Splitting for KKT System

- It simplifies to:

$$\begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ -2\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}$$

- Component-wise form:

$$\begin{aligned} \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^k &= \alpha \mathbf{x}^{k+1}, \\ -\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^k + \beta \mathbf{s}^k &= -2\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^{k+1} + \beta \mathbf{s}^{k+1} \end{aligned}$$

- Let $\mathbf{t}^k = \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^k$, then:

$$\begin{aligned} \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k &= \alpha \mathbf{x}^{k+1}, \\ -(\mathbf{I} - \mathbf{W}) \mathbf{x}^k + \beta \mathbf{t}^k &= -2(\mathbf{I} - \mathbf{W}) \mathbf{x}^{k+1} + \beta \mathbf{t}^{k+1} \end{aligned}$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\begin{aligned}\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k &= \alpha \mathbf{x}^{k+1}, \\ -(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta \mathbf{t}^k &= -2(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1} + \beta \mathbf{t}^{k+1}\end{aligned}$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$-(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta \mathbf{t}^k = -2(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1} + \beta \mathbf{t}^{k+1}$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Substitute $\mathbf{t}^k$ into the first equation:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \mathbf{t}^{k-1}$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Substitute $\mathbf{t}^k$ into the first equation:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \mathbf{t}^{k-1}$$

- Now express $\mathbf{t}^{k-1}$ from earlier iteration:

$$\mathbf{t}^{k-1} = \alpha \mathbf{x}^{k-1} - \nabla f(\mathbf{x}^{k-1}) - \alpha \mathbf{x}^k$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Substitute $\mathbf{t}^k$ into the first equation:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \mathbf{t}^{k-1}$$

- Now express $\mathbf{t}^{k-1}$ from earlier iteration:

$$\mathbf{t}^{k-1} = \alpha \mathbf{x}^{k-1} - \nabla f(\mathbf{x}^{k-1}) - \alpha \mathbf{x}^k$$

- Plug in $\mathbf{t}^{k-1}$:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \alpha \mathbf{x}^{k-1} + \nabla f(\mathbf{x}^{k-1}) + \alpha \mathbf{x}^k$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Plug in $\mathbf{t}^{k-1}$:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \alpha \mathbf{x}^{k-1} + \nabla f(\mathbf{x}^{k-1}) + \alpha \mathbf{x}^k$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Plug in $\mathbf{t}^{k-1}$:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \alpha \mathbf{x}^{k-1} + \nabla f(\mathbf{x}^{k-1}) + \alpha \mathbf{x}^k$$

- Simplify:

$$\alpha \mathbf{x}^{k+1} = \alpha(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \nabla f(\mathbf{x}^k) + \nabla f(\mathbf{x}^{k-1}) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1})$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Plug in $\mathbf{t}^{k-1}$:

$$\alpha \mathbf{x}^{k+1} = \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \alpha \mathbf{x}^{k-1} + \nabla f(\mathbf{x}^{k-1}) + \alpha \mathbf{x}^k$$

- Simplify:

$$\alpha \mathbf{x}^{k+1} = \alpha(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \nabla f(\mathbf{x}^k) + \nabla f(\mathbf{x}^{k-1}) - \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1})$$

- Combine terms:

$$\alpha \mathbf{x}^{k+1} = \left(\alpha \mathbf{I} - \frac{\mathbf{I} - \mathbf{W}}{\beta} \right) (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \nabla f(\mathbf{x}^k) + \nabla f(\mathbf{x}^{k-1})$$

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Combine terms:

$$\alpha \mathbf{x}^{k+1} = \left(\alpha \mathbf{I} - \frac{\mathbf{I} - \mathbf{W}}{\beta} \right) (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \nabla f(\mathbf{x}^k) + \nabla f(\mathbf{x}^{k-1})$$

- Let $\alpha\beta = 2$, so that

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2}(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- This is the update rule for the **EXTRA algorithm** (Shi et al., 2015 SIOPT)

EXact firST-order Algorithm (EXTRA)

- From the previous slide:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^k = \alpha \mathbf{x}^{k+1},$$

$$\frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) + \mathbf{t}^{k-1} = \mathbf{t}^k$$

- Let $\alpha\beta = 2$, so that

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2}(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- This is the update rule for the **EXTRA algorithm** (Shi et al., 2015 SIOPT)
- When $\alpha\beta = 2$, the matrix

$$\begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix}$$

is positive definite, and its convergence can be obtained from forward-backward splitting.

Convergence Conditions for EXTRA: I

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Convergence Conditions for EXTRA: I

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- If $f = 0$, the update simplifies to a linear system:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

Convergence Conditions for EXTRA: I

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- If $f = 0$, the update simplifies to a linear system:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Let $\mathbf{I} + \mathbf{W} = \mathbf{U}\Sigma\mathbf{U}^\top$ be an eigen-decomposition. Then:

$$\begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \\ & \mathbf{U} \end{bmatrix} \begin{bmatrix} \Sigma & -\frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top & \\ & \mathbf{U}^\top \end{bmatrix}$$

Convergence Conditions for EXTRA: I

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- If $f = 0$, the update simplifies to a linear system:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Let $\mathbf{I} + \mathbf{W} = \mathbf{U}\Sigma\mathbf{U}^\top$ be an eigen-decomposition. Then:

$$\begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \\ & \mathbf{U} \end{bmatrix} \begin{bmatrix} \Sigma & -\frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top & \\ & \mathbf{U}^\top \end{bmatrix}$$

- In the transformed coordinates:

$$\begin{bmatrix} \mathbf{U}^\top \mathbf{x}^{k+1} \\ \mathbf{U}^\top \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \Sigma & -\frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top \mathbf{x}^k \\ \mathbf{U}^\top \mathbf{x}^{k-1} \end{bmatrix}$$

Convergence Conditions for EXTRA: I

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- If $f = 0$, the update simplifies to a linear system:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Let $\mathbf{I} + \mathbf{W} = \mathbf{U}\Sigma\mathbf{U}^\top$ be an eigen-decomposition. Then:

$$\begin{bmatrix} \mathbf{I} + \mathbf{W} & -\frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \\ & \mathbf{U} \end{bmatrix} \begin{bmatrix} \Sigma & -\frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top & \\ & \mathbf{U}^\top \end{bmatrix}$$

- In the transformed coordinates:

$$\begin{bmatrix} \mathbf{U}^\top \mathbf{x}^{k+1} \\ \mathbf{U}^\top \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \Sigma & -\frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top \mathbf{x}^k \\ \mathbf{U}^\top \mathbf{x}^{k-1} \end{bmatrix}$$

- To ensure convergence for all other directions, the remaining eigenvalues of $\mathbf{W}$ must satisfy:

$$-\frac{2}{3} < \lambda(\mathbf{W} + \mathbf{I}) < 2 \quad \text{or equivalently} \quad -\frac{5}{3} < \lambda(\mathbf{W}) < 1$$

with **exact right equality allowed only at $\lambda = 1$ for consensus.**

Convergence Conditions for EXTRA: II

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Convergence Conditions for EXTRA: II

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- Suppose $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, i.e., $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{x} - \mathbf{b}\|^2$. Then the update becomes linear:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

Convergence Conditions for EXTRA: II

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- Suppose $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, i.e., $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{x} - \mathbf{b}\|^2$. Then the update becomes linear:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Diagonalize $\mathbf{I} + \mathbf{W} = \mathbf{U} \Sigma \mathbf{U}^\top$. The system becomes:

$$\begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \\ & \mathbf{U} \end{bmatrix} \begin{bmatrix} \Sigma - \frac{1}{\alpha} \mathbf{I} & -\frac{\Sigma}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top & \\ & \mathbf{U}^\top \end{bmatrix}$$

Convergence Conditions for EXTRA: II

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- Suppose $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, i.e., $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{x} - \mathbf{b}\|^2$. Then the update becomes linear:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Diagonalize $\mathbf{I} + \mathbf{W} = \mathbf{U} \Sigma \mathbf{U}^\top$. The system becomes:

$$\begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \\ & \mathbf{U} \end{bmatrix} \begin{bmatrix} \Sigma - \frac{1}{\alpha} \mathbf{I} & -\frac{\Sigma}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top & \\ & \mathbf{U}^\top \end{bmatrix}$$

- This leads to the condition:

$$\frac{4}{3\alpha} - \frac{2}{3} < \lambda(\mathbf{W} + \mathbf{I}) \leq 2 \iff \frac{4}{3\alpha} - \frac{5}{3} < \lambda(\mathbf{W}) \leq 1$$

Convergence Conditions for EXTRA: II

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

- Suppose $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, i.e., $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{x} - \mathbf{b}\|^2$. Then the update becomes linear:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Diagonalize $\mathbf{I} + \mathbf{W} = \mathbf{U} \Sigma \mathbf{U}^\top$. The system becomes:

$$\begin{bmatrix} \mathbf{I} + \mathbf{W} - \frac{1}{\alpha} \mathbf{I} & -\frac{\mathbf{I} + \mathbf{W}}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{U} & \\ & \mathbf{U} \end{bmatrix} \begin{bmatrix} \Sigma - \frac{1}{\alpha} \mathbf{I} & -\frac{\Sigma}{2} + \frac{1}{\alpha} \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top & \\ & \mathbf{U}^\top \end{bmatrix}$$

- This leads to the condition:

$$\frac{4}{3\alpha} - \frac{2}{3} < \lambda(\mathbf{W} + \mathbf{I}) \leq 2 \iff \frac{4}{3\alpha} - \frac{5}{3} < \lambda(\mathbf{W}) \leq 1$$

- Additionally, the stepsize must satisfy ($L = 1$):

$$\frac{1}{\alpha} < \frac{3}{4} \lambda_{\min}(\mathbf{W}) + \frac{5}{4} < \frac{2}{L}$$

Conditions for General EXTRA

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Conditions for General EXTRA

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Initialization (for $k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Conditions for General EXTRA

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Initialization (for $k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Convergence conditions (Li–Yan, 2021a Adv. Comput. Math.):

$$\frac{4}{3\alpha} - \frac{5}{3} < \lambda(\mathbf{W}) \leq 1$$
$$\frac{1}{\alpha} < \frac{2}{L}$$

Conditions for General EXTRA

EXTRA update:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Initialization (for $k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Convergence conditions (Li-Yan, 2021a Adv. Comput. Math.):

$$\frac{4}{3\alpha} - \frac{5}{3} < \lambda(\mathbf{W}) \leq 1$$
$$\frac{1}{\alpha} < \frac{2}{L}$$

Linear convergence requires:

- The global objective $\bar{f}(x) = \sum_{i=1}^n f_i(x)$ is strongly convex (Shi et al., 2015; Li-Yan, 2021b JOTA)

Decentralized Nonsmooth Optimization

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}$$

Decentralized Nonsmooth Optimization

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}$$

- Optimality conditions (KKT):

$$\mathbf{0} \in \nabla f(\mathbf{x}^*) + \partial g(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*$$

$$\mathbf{0} = -\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*$$

Decentralized Nonsmooth Optimization

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}$$

- Optimality conditions (KKT):

$$\mathbf{0} \in \nabla f(\mathbf{x}^*) + \partial g(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*$$

$$\mathbf{0} = -\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*$$

- Forward-backward splitting (primal-dual iteration):

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ \in & \begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} + \begin{bmatrix} \partial g & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} \end{aligned}$$

Decentralized Nonsmooth Optimization

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}$$

- Optimality conditions (KKT):

$$\mathbf{0} \in \nabla f(\mathbf{x}^*) + \partial g(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*$$

$$\mathbf{0} = -\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*$$

- Forward-backward splitting (primal-dual iteration):

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ \in & \begin{bmatrix} \alpha \mathbf{I} & -\sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} + \begin{bmatrix} \partial g & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} \end{aligned}$$

- Update equations:

$$\begin{aligned} \alpha \mathbf{x}^k - \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^k - \nabla f(\mathbf{x}^k) & \in \alpha \mathbf{x}^{k+1} + \partial g(\mathbf{x}^{k+1}) \\ -\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^k + \beta \mathbf{s}^k & = \beta \mathbf{s}^{k+1} - 2\sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^{k+1} \end{aligned}$$

PG-EXTRA

- For simplicity, let $\mathbf{t} = \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{s}$, then the iteration becomes:

$$\begin{aligned}\alpha \mathbf{x}^k - \mathbf{t}^k - \nabla f(\mathbf{x}^k) &\in \alpha \mathbf{x}^{k+1} + \partial g(\mathbf{x}^{k+1}) \\ -(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta \mathbf{t}^k &= \beta \mathbf{t}^{k+1} - 2(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1}\end{aligned}$$

PG-EXTRA

- For simplicity, let $\mathbf{t} = \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{s}$, then the iteration becomes:

$$\begin{aligned}\alpha\mathbf{x}^k - \mathbf{t}^k - \nabla f(\mathbf{x}^k) &\in \alpha\mathbf{x}^{k+1} + \partial g(\mathbf{x}^{k+1}) \\ -(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta\mathbf{t}^k &= \beta\mathbf{t}^{k+1} - 2(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1}\end{aligned}$$

- Define $\mathbf{z}^k := \mathbf{x}^{k-1} - \frac{1}{\alpha}\mathbf{t}^{k-1} - \frac{1}{\alpha}\nabla f(\mathbf{x}^{k-1})$. Then:

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{x}^k - \frac{1}{\alpha}\mathbf{t}^k - \frac{1}{\alpha}\nabla f(\mathbf{x}^k) \\ &= \mathbf{x}^k - \frac{1}{\alpha}\mathbf{t}^{k-1} - \frac{1}{\alpha} \cdot \frac{1}{\beta}(\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha}\nabla f(\mathbf{x}^k) \\ &= \mathbf{x}^k + \mathbf{z}^k - \mathbf{x}^{k-1} + \frac{1}{\alpha}\nabla f(\mathbf{x}^{k-1}) - \frac{\mathbf{I} - \mathbf{W}}{\alpha\beta}(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha}\nabla f(\mathbf{x}^k)\end{aligned}$$

PG-EXTRA

- For simplicity, let $\mathbf{t} = \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{s}$, then the iteration becomes:

$$\begin{aligned}\alpha \mathbf{x}^k - \mathbf{t}^k - \nabla f(\mathbf{x}^k) &\in \alpha \mathbf{x}^{k+1} + \partial g(\mathbf{x}^{k+1}) \\ -(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta \mathbf{t}^k &= \beta \mathbf{t}^{k+1} - 2(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1}\end{aligned}$$

- Define $\mathbf{z}^k := \mathbf{x}^{k-1} - \frac{1}{\alpha} \mathbf{t}^{k-1} - \frac{1}{\alpha} \nabla f(\mathbf{x}^{k-1})$. Then:

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{x}^k - \frac{1}{\alpha} \mathbf{t}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) \\ &= \mathbf{x}^k - \frac{1}{\alpha} \mathbf{t}^{k-1} - \frac{1}{\alpha} \cdot \frac{1}{\beta} (\mathbf{I} - \mathbf{W})(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) \\ &= \mathbf{x}^k + \mathbf{z}^k - \mathbf{x}^{k-1} + \frac{1}{\alpha} \nabla f(\mathbf{x}^{k-1}) - \frac{\mathbf{I} - \mathbf{W}}{\alpha\beta} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} \nabla f(\mathbf{x}^k)\end{aligned}$$

- Let $\alpha\beta = 2$, we arrive at the PG-EXTRA update (Shi et al. 2015b TSP):

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}\end{aligned}$$

Convergence Conditions for PG-EXTRA

PG-EXTRA iterations:

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}\end{aligned}$$

Convergence Conditions for PG-EXTRA

PG-EXTRA iterations:

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}\end{aligned}$$

Convergence conditions (Li-Yan, 2021a):

$$\begin{aligned}\frac{4L}{3\alpha} - \frac{5}{3} &< \lambda(\mathbf{W}) \leq 1, \\ \frac{1}{\alpha} &< \frac{3}{L} \lambda_{\min}(\mathbf{W}) + \frac{5}{4L} < \frac{2}{L}\end{aligned}$$

Convergence Conditions for PG-EXTRA

PG-EXTRA iterations:

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}\end{aligned}$$

Convergence conditions (Li-Yan, 2021a):

$$\begin{aligned}\frac{4L}{3\alpha} - \frac{5}{3} &< \lambda(\mathbf{W}) \leq 1, \\ \frac{1}{\alpha} &< \frac{3}{L} \lambda_{\min}(\mathbf{W}) + \frac{5}{4L} < \frac{2}{L}\end{aligned}$$

Initialization (for $k = 0, 1$):

$$\begin{aligned}\mathbf{z}^1 &= \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0), \\ \mathbf{x}^1 &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^1\end{aligned}$$

large stepsize as centralized ones?

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

- **Lagrangian:**

$$\mathcal{L}(\mathbf{x}, \mathbf{s}) = f(\mathbf{x}) + \langle \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}, \mathbf{s} \rangle,$$

where $\mathbf{s}$ is the Lagrange multiplier.

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

- **Lagrangian:**

$$\mathcal{L}(\mathbf{x}, \mathbf{s}) = f(\mathbf{x}) + \langle \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}, \mathbf{s} \rangle,$$

where $\mathbf{s}$ is the Lagrange multiplier.

- **KKT Conditions (Optimality):**

$$\mathbf{0} = \nabla f(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*,$$

$$\mathbf{0} = \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*.$$

Decentralized Smooth Optimization

Problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{subject to} \quad \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x} = \mathbf{0}.$$

- **Lagrangian:**

$$\mathcal{L}(\mathbf{x}, \mathbf{s}) = f(\mathbf{x}) + \langle \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}, \mathbf{s} \rangle,$$

where $\mathbf{s}$ is the Lagrange multiplier.

- **KKT Conditions (Optimality):**

$$\begin{aligned} \mathbf{0} &= \nabla f(\mathbf{x}^*) + \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^*, \\ \mathbf{0} &= \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^*. \end{aligned}$$

- **Compact Matrix Form:**

$$-\begin{bmatrix} \nabla f(\mathbf{x}^*) \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^* \\ \mathbf{s}^* \end{bmatrix}.$$

Forward-Backward Splitting (Another Form)

- The KKT system:

$$= - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} + \begin{bmatrix} \mathbf{0} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}.$$

Forward-Backward Splitting (Another Form)

- Apply forward-backward splitting using a diagonal preconditioning:

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ = & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix} + \begin{bmatrix} \mathbf{0} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}. \end{aligned}$$

Forward-Backward Splitting (Another Form)

- Combine the right-hand side into a single linear operator:

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ = & \begin{bmatrix} \alpha \mathbf{I} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}. \end{aligned}$$

Forward-Backward Splitting (Another Form)

- Combine the right-hand side into a single linear operator:

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ &= \begin{bmatrix} \alpha \mathbf{I} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}. \end{aligned}$$

- Apply Gaussian elimination (eliminate $\mathbf{x}^{k+1}$ from the second row):

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \frac{1}{\alpha} \sqrt{\mathbf{I} - \mathbf{W}} \nabla f(\mathbf{x}^k) \end{bmatrix} \\ &= \begin{bmatrix} \alpha \mathbf{I} & \sqrt{\mathbf{I} - \mathbf{W}} \\ \mathbf{0} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}. \end{aligned}$$

Forward-Backward Splitting (Another Form)

- Combine the right-hand side into a single linear operator:

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \mathbf{0} \end{bmatrix} \\ &= \begin{bmatrix} \alpha \mathbf{I} & \sqrt{\mathbf{I} - \mathbf{W}} \\ -\sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}. \end{aligned}$$

- Apply Gaussian elimination (eliminate $\mathbf{x}^{k+1}$ from the second row):

$$\begin{aligned} & \begin{bmatrix} \alpha \mathbf{I} & \mathbf{0} \\ \sqrt{\mathbf{I} - \mathbf{W}} & \beta \mathbf{I} - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W}) \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{s}^k \end{bmatrix} - \begin{bmatrix} \nabla f(\mathbf{x}^k) \\ \frac{1}{\alpha} \sqrt{\mathbf{I} - \mathbf{W}} \nabla f(\mathbf{x}^k) \end{bmatrix} \\ &= \begin{bmatrix} \alpha \mathbf{I} & \sqrt{\mathbf{I} - \mathbf{W}} \\ \mathbf{0} & \beta \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{s}^{k+1} \end{bmatrix}. \end{aligned}$$

- Final update equations:

$$\begin{aligned} \alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^{k+1} &= \alpha \mathbf{x}^{k+1}, \\ \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^k + \beta \left(\mathbf{I} - \frac{1}{\alpha \beta} (\mathbf{I} - \mathbf{W}) \right) \mathbf{s}^k - \frac{1}{\alpha} \sqrt{\mathbf{I} - \mathbf{W}} \nabla f(\mathbf{x}^k) &= \beta \mathbf{s}^{k+1}. \end{aligned}$$

NIDS (Li-Shi-Yan, 2019 TSP)

From the previous slide:

$$\begin{aligned}\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^{k+1} &= \alpha \mathbf{x}^{k+1}, \\ \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^k + \beta \left(\mathbf{I} - \frac{1}{\alpha\beta} (\mathbf{I} - \mathbf{W}) \right) \mathbf{s}^k - \frac{1}{\alpha} \sqrt{\mathbf{I} - \mathbf{W}} \nabla f(\mathbf{x}^k) &= \beta \mathbf{s}^{k+1}.\end{aligned}$$

NIDS (Li-Shi-Yan, 2019 TSP)

From the previous slide:

$$\begin{aligned}\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}^{k+1} &= \alpha \mathbf{x}^{k+1}, \\ \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{x}^k + \beta \left(\mathbf{I} - \frac{1}{\alpha\beta} (\mathbf{I} - \mathbf{W}) \right) \mathbf{s}^k - \frac{1}{\alpha} \sqrt{\mathbf{I} - \mathbf{W}} \nabla f(\mathbf{x}^k) &= \beta \mathbf{s}^{k+1}.\end{aligned}$$

Let $\mathbf{t} = \sqrt{\mathbf{I} - \mathbf{W}} \mathbf{s}$, then:

$$\begin{aligned}\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^{k+1} &= \alpha \mathbf{x}^{k+1}, \\ (\mathbf{I} - \mathbf{W}) \mathbf{x}^k + \beta \left(\mathbf{I} - \frac{1}{\alpha\beta} (\mathbf{I} - \mathbf{W}) \right) \mathbf{t}^k - \frac{1}{\alpha} (\mathbf{I} - \mathbf{W}) \nabla f(\mathbf{x}^k) &= \beta \mathbf{t}^{k+1}.\end{aligned}$$

NIDS (Li-Shi-Yan, 2019 TSP)

Let $\mathbf{t} = \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{s}$, then:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^{k+1} = \alpha \mathbf{x}^{k+1},$$

$$(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \mathbf{t}^k - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W})\nabla f(\mathbf{x}^k) = \beta \mathbf{t}^{k+1}.$$

Now expand $\mathbf{t}^{k+1}$ and plug it into the first line:

$$\begin{aligned} \mathbf{x}^{k+1} &= \mathbf{x}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) - \frac{1}{\alpha} \mathbf{t}^{k+1} \\ &= \mathbf{x}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) - \frac{1}{\alpha} \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \mathbf{t}^k - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \frac{1}{\alpha} \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W})\nabla f(\mathbf{x}^k) \\ &= \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \left(\mathbf{x}^k - \frac{1}{\alpha} \mathbf{t}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) \right). \end{aligned}$$

NIDS (Li-Shi-Yan, 2019 TSP)

Let $\mathbf{t} = \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{s}$, then:

$$\alpha \mathbf{x}^k - \nabla f(\mathbf{x}^k) - \mathbf{t}^{k+1} = \alpha \mathbf{x}^{k+1},$$

$$(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \beta \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \mathbf{t}^k - \frac{1}{\alpha}(\mathbf{I} - \mathbf{W})\nabla f(\mathbf{x}^k) = \beta \mathbf{t}^{k+1}.$$

Now expand $\mathbf{t}^{k+1}$ and plug it into the first line:

$$\begin{aligned} \mathbf{x}^{k+1} &= \mathbf{x}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) - \frac{1}{\alpha} \mathbf{t}^{k+1} \\ &= \mathbf{x}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) - \frac{1}{\alpha} \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \mathbf{t}^k - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W})\mathbf{x}^k + \frac{1}{\alpha} \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W})\nabla f(\mathbf{x}^k) \\ &= \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \left(\mathbf{x}^k - \frac{1}{\alpha} \mathbf{t}^k - \frac{1}{\alpha} \nabla f(\mathbf{x}^k) \right). \end{aligned}$$

Recall that

$$-\frac{1}{\alpha} \mathbf{t}^k = \mathbf{x}^k - \mathbf{x}^{k-1} + \frac{1}{\alpha} \nabla f(\mathbf{x}^{k-1}),$$

so:

$$\mathbf{x}^{k+1} = \left(\mathbf{I} - \frac{1}{\alpha\beta}(\mathbf{I} - \mathbf{W}) \right) \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right).$$

Convergence Conditions for NIDS

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

Convergence Conditions for NIDS

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- If $f = 0$ (same as EXTRA): the condition for $\mathbf{W}$ is

$$-\frac{5}{3} < \lambda(\mathbf{W}) \leq 1.$$

Convergence Conditions for NIDS

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- If $f = 0$ (same as EXTRA): the condition for $\mathbf{W}$ is

$$-\frac{5}{3} < \lambda(\mathbf{W}) \leq 1.$$

- If $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, then the iteration becomes:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} (2 - \frac{1}{\alpha}) \frac{\mathbf{I} + \mathbf{W}}{2} & -(1 - \frac{1}{\alpha}) \frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

Convergence Conditions for NIDS

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- If $f = 0$ (same as EXTRA): the condition for $\mathbf{W}$ is

$$-\frac{5}{3} < \lambda(\mathbf{W}) \leq 1.$$

- If $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, then the iteration becomes:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} (2 - \frac{1}{\alpha}) \frac{\mathbf{I} + \mathbf{W}}{2} & -(1 - \frac{1}{\alpha}) \frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Let $\mathbf{I} + \mathbf{W} = \mathbf{U}\Sigma\mathbf{U}^\top$, then the system becomes diagonalized:

$$\begin{bmatrix} \mathbf{U}^\top \mathbf{x}^{k+1} \\ \mathbf{U}^\top \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} (2 - \frac{1}{\alpha}) \frac{\Sigma}{2} & -(1 - \frac{1}{\alpha}) \frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top \mathbf{x}^k \\ \mathbf{U}^\top \mathbf{x}^{k-1} \end{bmatrix}$$

Convergence Conditions for NIDS

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- If $f = 0$ (same as EXTRA): the condition for $\mathbf{W}$ is

$$-\frac{5}{3} < \lambda(\mathbf{W}) \leq 1.$$

- If $\nabla f(\mathbf{x}^k) = \mathbf{x}^k - \mathbf{b}$, then the iteration becomes:

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} (2 - \frac{1}{\alpha}) \frac{\mathbf{I} + \mathbf{W}}{2} & -(1 - \frac{1}{\alpha}) \frac{\mathbf{I} + \mathbf{W}}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^k \\ \mathbf{x}^{k-1} \end{bmatrix}$$

- Let $\mathbf{I} + \mathbf{W} = \mathbf{U}\Sigma\mathbf{U}^\top$, then the system becomes diagonalized:

$$\begin{bmatrix} \mathbf{U}^\top \mathbf{x}^{k+1} \\ \mathbf{U}^\top \mathbf{x}^k \end{bmatrix} = \begin{bmatrix} (2 - \frac{1}{\alpha}) \frac{\Sigma}{2} & -(1 - \frac{1}{\alpha}) \frac{\Sigma}{2} \\ \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{U}^\top \mathbf{x}^k \\ \mathbf{U}^\top \mathbf{x}^{k-1} \end{bmatrix}$$

- Therefore, the convergence condition is:

$$-\frac{5}{3} < \lambda(\mathbf{W}) \leq 1 \quad \text{and} \quad \frac{1}{\alpha} < 2.$$

Conditions of NIDS for General Smooth Functions

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

Conditions of NIDS for General Smooth Functions

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

Initial condition ($k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Conditions of NIDS for General Smooth Functions

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

Initial condition ($k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Convergence condition (Li-Yan, 2021a):

$$\begin{aligned} -\frac{5}{3} &< \lambda(\mathbf{W}) \leq 1, \\ \frac{1}{\alpha} &< \frac{2}{L}. \end{aligned}$$

Conditions of NIDS for General Smooth Functions

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

Initial condition ($k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Convergence condition (Li-Yan, 2021a):

$$\begin{aligned} -\frac{5}{3} &< \lambda(\mathbf{W}) \leq 1, \\ \frac{1}{\alpha} &< \frac{2}{L}. \end{aligned}$$

Linear convergence conditions:

- If $f(\mathbf{x})$ is strongly convex and $-1 < \lambda(\mathbf{W}) \leq 1$ (Li-Shi-Yan, 2019):

$$\text{Rate: } O \left(\max \left(1 - \frac{\mu}{L}, 1 - \frac{1 - \lambda_2(\mathbf{W})}{1 - \lambda_n(\mathbf{W})} \right) \right)$$

Conditions of NIDS for General Smooth Functions

NIDS (with $\alpha\beta = 2$):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

Initial condition ($k = 0, 1$):

$$\mathbf{x}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0)$$

Convergence condition (Li-Yan, 2021a):

$$\begin{aligned} -\frac{5}{3} &< \lambda(\mathbf{W}) \leq 1, \\ \frac{1}{\alpha} &< \frac{2}{L}. \end{aligned}$$

Linear convergence conditions:

- If $f(\mathbf{x})$ is strongly convex and $-1 < \lambda(\mathbf{W}) \leq 1$ (Li-Shi-Yan, 2019):

$$\text{Rate: } O \left(\max \left(1 - \frac{\mu}{L}, 1 - \frac{1 - \lambda_2(\mathbf{W})}{1 - \lambda_n(\mathbf{W})} \right) \right)$$

- If $\bar{f}(x)$ is strongly convex and $-\frac{5}{3} < \lambda(\mathbf{W}) \leq 1$ (Li-Yan, 2021b).

NIDS vs EXTRA

EXTRA:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} \right) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

NIDS:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

NIDS vs EXTRA

EXTRA:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

NIDS:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- The key difference is the placement of the gradient term inside vs. outside the mixing operator.

NIDS vs EXTRA

EXTRA:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

NIDS:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- The key difference is the placement of the gradient term inside vs. outside the mixing operator.
- This results in different communication and computation patterns:
 - EXTRA can apply gradient differences during consensus mixing.
 - NIDS mixes after applying gradient differences.

NIDS vs EXTRA

EXTRA:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

NIDS:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- The key difference is the placement of the gradient term inside vs. outside the mixing operator.
- This results in different communication and computation patterns:
 - EXTRA can apply gradient differences during consensus mixing.
 - NIDS mixes after applying gradient differences.
- NIDS allows for a **larger step size range** than EXTRA.

NIDS vs EXTRA

EXTRA:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} \right) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

NIDS:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

- The key difference is the placement of the gradient term inside vs. outside the mixing operator.
- This results in different communication and computation patterns:
 - EXTRA can apply gradient differences during consensus mixing.
 - NIDS mixes after applying gradient differences.
- NIDS allows for a **larger step size range** than EXTRA.
- Empirically and theoretically, **NIDS converges faster** than EXTRA under the same settings.

NIDS for Nonsmooth Functions

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

NIDS for Nonsmooth Functions

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

PG-EXTRA:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2}(2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha}\partial g\right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

NIDS for Nonsmooth Functions

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

PG-EXTRA:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

NIDS:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

NIDS for Nonsmooth Functions

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

PG-EXTRA:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

NIDS:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

- The key difference is the placement of the gradient difference term:
 - PG-EXTRA applies it **outside** the mixing step.
 - NIDS applies it **inside** the mixing step.

NIDS for Nonsmooth Functions

Problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) + g(\mathbf{x}), \quad \text{s.t.} \quad \sqrt{\mathbf{I} - \mathbf{W}}\mathbf{x} = \mathbf{0}.$$

PG-EXTRA:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} (2\mathbf{x}^k - \mathbf{x}^{k-1}) - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

NIDS:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right), \\ \mathbf{x}^{k+1} &= \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}. \end{aligned}$$

- The key difference is the placement of the gradient difference term:
 - PG-EXTRA applies it **outside** the mixing step.
 - NIDS applies it **inside** the mixing step.
- As in the smooth case, NIDS allows a wider step size range and often achieves better empirical performance.

Convergence Conditions for NIDS (Nonsmooth)

NIDS iteration:

$$\begin{aligned}\mathbf{z}^{k+1} &= \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right), \\ \mathbf{x}^{k+1} &= \text{prox}_{(1/\alpha)g}(\mathbf{z}^{k+1}) = \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}.\end{aligned}$$

Convergence Conditions for NIDS (Nonsmooth)

NIDS iteration:

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right),$$
$$\mathbf{x}^{k+1} = \text{prox}_{(1/\alpha)g}(\mathbf{z}^{k+1}) = \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}.$$

Convergence Conditions (Li-Shi-Yan, 2019):

$-1 < \lambda(\mathbf{W}) \leq 1$, Open Question: can the lower bound be $-5/3$?

$$\frac{1}{\alpha} < 2/L.$$

Convergence Conditions for NIDS (Nonsmooth)

NIDS iteration:

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right),$$
$$\mathbf{x}^{k+1} = \text{prox}_{(1/\alpha)g}(\mathbf{z}^{k+1}) = \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}.$$

Convergence Conditions (Li-Shi-Yan, 2019):

$$-1 < \lambda(\mathbf{W}) \leq 1, \quad \text{Open Question: can the lower bound be } -5/3?$$

$$\frac{1}{\alpha} < 2/L.$$

Initialization:

$$\mathbf{z}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0),$$
$$\mathbf{x}^1 = \text{prox}_{(1/\alpha)g}(\mathbf{z}^1) = \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^1.$$

Convergence Conditions for NIDS (Nonsmooth)

NIDS iteration:

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \mathbf{x}^k + \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right),$$
$$\mathbf{x}^{k+1} = \text{prox}_{(1/\alpha)g}(\mathbf{z}^{k+1}) = \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^{k+1}.$$

Convergence Conditions (Li-Shi-Yan, 2019):

$$-1 < \lambda(\mathbf{W}) \leq 1, \quad \text{Open Question: can the lower bound be } -5/3?$$

$$\frac{1}{\alpha} < 2/L.$$

Initialization:

$$\mathbf{z}^1 = \mathbf{x}^0 - \frac{1}{\alpha} \nabla f(\mathbf{x}^0),$$
$$\mathbf{x}^1 = \text{prox}_{(1/\alpha)g}(\mathbf{z}^1) = \left(\mathbf{I} + \frac{1}{\alpha} \partial g \right)^{-1} \mathbf{z}^1.$$

Remarks:

- Assumes f is smooth and L -Lipschitz gradient.
- Allows nonsmooth but proximable g .

- Step size α can be chosen based on local L_i .

Advantages of NIDS

NIDS iteration:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right).$$

Advantages of NIDS

NIDS iteration:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right).$$

- Stepsize is large and independent of network topology:

$$\frac{1}{\alpha} < \frac{2}{L}.$$

Advantages of NIDS

NIDS iteration:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right).$$

- **Stepsize is large and independent of network topology:**

$$\frac{1}{\alpha} < \frac{2}{L}.$$

- **Supports individual (node-dependent) stepsizes:**

$$\frac{1}{\alpha_i} < \frac{2}{L_i}, \quad \text{for each node } i.$$

Advantages of NIDS

NIDS iteration:

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right).$$

- **Stepsize is large and independent of network topology:**

$$\frac{1}{\alpha} < \frac{2}{L}.$$

- **Supports individual (node-dependent) stepsizes:**

$$\frac{1}{\alpha_i} < \frac{2}{L_i}, \quad \text{for each node } i.$$

- **Linear convergence with decoupled rates:**

$$O \left(\max \left(1 - \frac{\mu}{L}, 1 - \frac{1 - \lambda_2(\mathbf{W})}{1 - \lambda_n(\mathbf{W})} \right) \right).$$

- Matches centralized gradient descent rate: $1 - \mu/L$.
- Matches decentralized averaging rate: $1 - (1 - \lambda_2)/(1 - \lambda_n)$.

D²: Stochastic Nonconvex NIDS (Huang et al., 2018 ICML)

NIDS (deterministic):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

D²: Stochastic Nonconvex NIDS (Huang et al., 2018 ICML)

NIDS (deterministic):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

D² (Decentralized Training over Decentralized Data):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k, \xi_k) - \nabla f(\mathbf{x}^{k-1}, \xi_{k-1})) \right)$$

D²: Stochastic Nonconvex NIDS (Huang et al., 2018 ICML)

NIDS (deterministic):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

D² (Decentralized Training over Decentralized Data):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k, \xi_k) - \nabla f(\mathbf{x}^{k-1}, \xi_{k-1})) \right)$$

- $\nabla f(\mathbf{x}^k, \xi_k)$ is a stochastic gradient sampled with $\xi_k \sim \mathcal{D}$.

D²: Stochastic Nonconvex NIDS (Huang et al., 2018 ICML)

NIDS (deterministic):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

D² (Decentralized Training over Decentralized Data):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k, \xi_k) - \nabla f(\mathbf{x}^{k-1}, \xi_{k-1})) \right)$$

- $\nabla f(\mathbf{x}^k, \xi_k)$ is a stochastic gradient sampled with $\xi_k \sim \mathcal{D}$.
- **Unbiasedness:**

$$\mathbb{E}_{\xi \sim \mathcal{D}} [\nabla f(\mathbf{x}; \xi)] = \nabla f(\mathbf{x}), \quad \forall \mathbf{x}.$$

D²: Stochastic Nonconvex NIDS (Huang et al., 2018 ICML)

NIDS (deterministic):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

D² (Decentralized Training over Decentralized Data):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k, \xi_k) - \nabla f(\mathbf{x}^{k-1}, \xi_{k-1})) \right)$$

- $\nabla f(\mathbf{x}^k, \xi_k)$ is a stochastic gradient sampled with $\xi_k \sim \mathcal{D}$.
- **Unbiasedness:**

$$\mathbb{E}_{\xi \sim \mathcal{D}} [\nabla f(\mathbf{x}; \xi)] = \nabla f(\mathbf{x}), \quad \forall \mathbf{x}.$$

- **Bounded variance:**

$$\mathbb{E}_{\xi \sim \mathcal{D}} \|\nabla f(\mathbf{x}; \xi) - \nabla f(\mathbf{x})\|^2 \leq \sigma^2, \quad \forall \mathbf{x}.$$

D²: Stochastic Nonconvex NIDS (Huang et al., 2018 ICML)

NIDS (deterministic):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1})) \right)$$

D² (Decentralized Training over Decentralized Data):

$$\mathbf{x}^{k+1} = \frac{\mathbf{I} + \mathbf{W}}{2} \left(2\mathbf{x}^k - \mathbf{x}^{k-1} - \frac{1}{\alpha} (\nabla f(\mathbf{x}^k, \xi_k) - \nabla f(\mathbf{x}^{k-1}, \xi_{k-1})) \right)$$

- $\nabla f(\mathbf{x}^k, \xi_k)$ is a stochastic gradient sampled with $\xi_k \sim \mathcal{D}$.
- **Unbiasedness:**

$$\mathbb{E}_{\xi \sim \mathcal{D}} [\nabla f(\mathbf{x}; \xi)] = \nabla f(\mathbf{x}), \quad \forall \mathbf{x}.$$

- **Bounded variance:**

$$\mathbb{E}_{\xi \sim \mathcal{D}} \|\nabla f(\mathbf{x}; \xi) - \nabla f(\mathbf{x})\|^2 \leq \sigma^2, \quad \forall \mathbf{x}.$$

- **Convergence rate:** For sufficiently small stepsize $\alpha = O\left(\frac{1}{c + \sqrt{T/n}}\right)$,

$$\frac{1}{T} \sum_{t=0}^T \mathbb{E} \|\nabla \bar{f}(\bar{\mathbf{x}}_t)\|^2 = O\left(\frac{\sigma}{\sqrt{nT}} + \frac{1}{T}\right).$$

ABC / UDA Algorithm (Xu et al., 2021; Alghunaim et al., 2021)

General Form:

$$\mathbf{x}^{k+1} = (\mathbf{I} - \mathbf{C})\mathbf{x}^k - \gamma \mathbf{A} \nabla f(\mathbf{x}^k) - \mathbf{y}^k,$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \mathbf{B}\mathbf{x}^{k+1}$$

ABC / UDA Algorithm (Xu et al., 2021; Alghunaim et al., 2021)

General Form:

$$\begin{aligned}\mathbf{x}^{k+1} &= (\mathbf{I} - \mathbf{C})\mathbf{x}^k - \gamma \mathbf{A} \nabla f(\mathbf{x}^k) - \mathbf{y}^k, \\ \mathbf{y}^{k+1} &= \mathbf{y}^k + \mathbf{B}\mathbf{x}^{k+1}\end{aligned}$$

One-Line Update:

$$\mathbf{x}^{k+1} = (\mathbf{I} - \mathbf{B})\mathbf{x}^k + (\mathbf{I} - \mathbf{C})(\mathbf{x}^k - \mathbf{x}^{k-1}) - \gamma \mathbf{A} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Special Cases:

- EXTRA: $\mathbf{A} = \mathbf{I}$, $\mathbf{B} = \mathbf{C} = \frac{\mathbf{I} - \mathbf{W}}{2}$
- NIDS/Exact Diffusion: $\mathbf{A} = \frac{\mathbf{I} + \mathbf{W}}{2}$, $\mathbf{B} = \mathbf{C} = \frac{\mathbf{I} - \mathbf{W}}{2}$

ABC / UDA Algorithm (Xu et al., 2021; Alghunaim et al., 2021)

General Form:

$$\begin{aligned}\mathbf{x}^{k+1} &= (\mathbf{I} - \mathbf{C})\mathbf{x}^k - \gamma\mathbf{A}\nabla f(\mathbf{x}^k) - \mathbf{y}^k, \\ \mathbf{y}^{k+1} &= \mathbf{y}^k + \mathbf{B}\mathbf{x}^{k+1}\end{aligned}$$

Two-Line Gradient Tracking Form:

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{W}_1\mathbf{x}^k - \gamma\mathbf{W}_2\mathbf{s}^k, \\ \mathbf{s}^{k+1} &= \mathbf{W}_3\mathbf{s}^k + \mathbf{W}_4 \left(\nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k) \right)\end{aligned}$$

Conditions for Equivalence:

$$\mathbf{W}_3\mathbf{W}_1 = \mathbf{I} - \mathbf{C}, \quad \mathbf{W}_1 + \mathbf{W}_3 - \mathbf{W}_3\mathbf{W}_1 = \mathbf{I} - \mathbf{B}, \quad \mathbf{W}_2\mathbf{W}_4 = \mathbf{A}$$

ABC / UDA Algorithm (Xu et al., 2021; Alghunaim et al., 2021)

General Form:

$$\begin{aligned}\mathbf{x}^{k+1} &= (\mathbf{I} - \mathbf{C})\mathbf{x}^k - \gamma \mathbf{A} \nabla f(\mathbf{x}^k) - \mathbf{y}^k, \\ \mathbf{y}^{k+1} &= \mathbf{y}^k + \mathbf{B}\mathbf{x}^{k+1}\end{aligned}$$

Two-Line Gradient Tracking Form:

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{W}_1 \mathbf{x}^k - \gamma \mathbf{W}_2 \mathbf{s}^k, \\ \mathbf{s}^{k+1} &= \mathbf{W}_3 \mathbf{s}^k + \mathbf{W}_4 (\nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k))\end{aligned}$$

Conditions for Equivalence:

$$\mathbf{W}_3 \mathbf{W}_1 = \mathbf{I} - \mathbf{C}, \quad \mathbf{W}_1 + \mathbf{W}_3 - \mathbf{W}_3 \mathbf{W}_1 = \mathbf{I} - \mathbf{B}, \quad \mathbf{W}_2 \mathbf{W}_4 = \mathbf{A}$$

- DIGing/Harnessing: $\mathbf{W}_1 = \mathbf{W}_3 = \mathbf{W}$, $\mathbf{W}_2 = \mathbf{W}_4 = \mathbf{I} \Rightarrow \mathbf{A} = \mathbf{I}$, $\mathbf{B} = (\mathbf{I} - \mathbf{W})^2$, $\mathbf{C} = \mathbf{I} - \mathbf{W}^2$
- ATC-Tracking: $\mathbf{W}_1 = \mathbf{W}_2 = \mathbf{W}_3 = \mathbf{W}$, $\mathbf{W}_4 = \mathbf{I} \Rightarrow \mathbf{A} = \mathbf{W}$, $\mathbf{B} = (\mathbf{I} - \mathbf{W})^2$, $\mathbf{C} = \mathbf{I} - \mathbf{W}^2$
- NEXT/Aug-DGM / ATC-DIGing: $\mathbf{W}_1 = \mathbf{W}_2 = \mathbf{W}_3 = \mathbf{W}_4 = \mathbf{W} \Rightarrow \mathbf{A} = \mathbf{W}^2$, $\mathbf{B} = (\mathbf{I} - \mathbf{W})^2$, $\mathbf{C} = \mathbf{I} - \mathbf{W}^2$

Outline

Introduction

Average Consensus

Decentralized algorithms

DGD

EXTRA/PG-EXTRA

NIDS

General framework

Accelerated algorithms

OGT: Optimal Gradient Tracking with Loopless Chebyshev Acceleration

OPAPC: Optimized PAPC with Chebyshev Acceleration

Algorithms with Compression

LEAD: Linear convergent decentralized algorithm with compression

Directed Network

APD

Gradient Tracking

Algorithm (*Initialization:* $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$):

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma\mathbf{s}^k,$$

$$\mathbf{s}^{k+1} = \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

Gradient Tracking

Algorithm (*Initialization*: $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$):

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma\mathbf{s}^k,$$

$$\mathbf{s}^{k+1} = \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

- $\mathbf{x}^k$ contains the local variables of all agents at iteration k .
- $\mathbf{s}^k$ tracks the average gradient across all agents.

Gradient Tracking

Algorithm (*Initialization:* $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$):

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma\mathbf{s}^k,$$

$$\mathbf{s}^{k+1} = \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

- $\mathbf{x}^k$ contains the local variables of all agents at iteration k .
- $\mathbf{s}^k$ tracks the average gradient across all agents.

Gradient Tracking Property:

$$\mathbf{1}^\top \mathbf{s}^{k+1} = \mathbf{1}^\top \mathbf{W}\mathbf{s}^k + \mathbf{1}^\top (\nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k))$$

$$= \mathbf{1}^\top \mathbf{s}^k + \mathbf{1}^\top \nabla f(\mathbf{x}^{k+1}) - \mathbf{1}^\top \nabla f(\mathbf{x}^k)$$

$$= \mathbf{1}^\top \nabla f(\mathbf{x}^{k+1})$$

$$\mathbf{1}^\top \mathbf{x}^{k+1} = \mathbf{1}^\top \mathbf{x}^k - \gamma \cdot \mathbf{1}^\top \nabla f(\mathbf{x}^k)$$

Gradient Tracking

Algorithm (*Initialization*: $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$):

$$\mathbf{x}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma\mathbf{s}^k,$$

$$\mathbf{s}^{k+1} = \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

- $\mathbf{x}^k$ contains the local variables of all agents at iteration k .
- $\mathbf{s}^k$ tracks the average gradient across all agents.

Gradient Tracking Property:

$$\begin{aligned}\mathbf{1}^\top \mathbf{s}^{k+1} &= \mathbf{1}^\top \mathbf{W}\mathbf{s}^k + \mathbf{1}^\top (\nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)) \\ &= \mathbf{1}^\top \mathbf{s}^k + \mathbf{1}^\top \nabla f(\mathbf{x}^{k+1}) - \mathbf{1}^\top \nabla f(\mathbf{x}^k) \\ &= \mathbf{1}^\top \nabla f(\mathbf{x}^{k+1}) \\ \mathbf{1}^\top \mathbf{x}^{k+1} &= \mathbf{1}^\top \mathbf{x}^k - \gamma \cdot \mathbf{1}^\top \nabla f(\mathbf{x}^k)\end{aligned}$$

Convergence Rate: If f is μ -strongly convex and L -smooth:

$$O\left(\left(\kappa + \frac{1}{\theta^2}\right) \log \frac{1}{\epsilon}\right) \quad (\text{computation and communication})$$

where $\kappa = L/\mu$ and θ is the spectral gap of $\mathbf{W}$.

Gradient Tracking: How Does It Work?

We consider the averaged quantities: $\bar{\mathbf{x}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{x}^k$ and $\bar{\mathbf{s}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{s}^k$.

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{W}\mathbf{x}^k - \gamma \mathbf{s}^k, \\ \mathbf{s}^{k+1} &= \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)\end{aligned}$$

Gradient Tracking: How Does It Work?

We consider the averaged quantities: $\bar{\mathbf{x}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{x}^k$ and $\bar{\mathbf{s}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{s}^k$.

$$\bar{\mathbf{x}}^{k+1} = \bar{\mathbf{x}}^k - \gamma \bar{\mathbf{s}}^k,$$

$$\bar{\mathbf{s}}^k = \frac{1}{n} \mathbf{1}^\top \nabla f(\mathbf{x}^k) \neq \nabla \bar{f}(\bar{\mathbf{x}}^k)$$

- $\mathbf{x}^k$ is not consensual, i.e., $\mathbf{x}^k \neq \mathbf{1} \bar{\mathbf{x}}^k$.
- We need to bound the error caused by gradient disagreement.

Gradient Tracking: How Does It Work?

We consider the averaged quantities: $\bar{\mathbf{x}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{x}^k$ and $\bar{\mathbf{s}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{s}^k$.

$$\bar{\mathbf{x}}^{k+1} = \bar{\mathbf{x}}^k - \gamma \bar{\mathbf{s}}^k,$$

$$\bar{\mathbf{s}}^k = \frac{1}{n} \mathbf{1}^\top \nabla f(\mathbf{x}^k) \neq \nabla \bar{f}(\bar{\mathbf{x}}^k)$$

- $\mathbf{x}^k$ is not consensual, i.e., $\mathbf{x}^k \neq \mathbf{1} \bar{\mathbf{x}}^k$.
- We need to bound the error caused by gradient disagreement.

We have:

$$f(\bar{\mathbf{x}}^k) - f(\mathbf{x}^*) \leq \langle \bar{\mathbf{s}}^k, \bar{\mathbf{x}}^k - \mathbf{x}^* \rangle - \frac{\mu}{4} \|\bar{\mathbf{x}}^k - \mathbf{x}^*\|^2 + \frac{L}{n} \|\Pi \mathbf{x}^k\|_F^2$$

Here, $\Pi \mathbf{x}^k = \mathbf{x}^k - \frac{1}{n} \mathbf{1} \mathbf{1}^\top \mathbf{x}^k$ measures the deviation from consensus.

Gradient Tracking: How Does It Work?

We consider the averaged quantities: $\bar{\mathbf{x}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{x}^k$ and $\bar{\mathbf{s}}^k = \frac{1}{n} \mathbf{1}^\top \mathbf{s}^k$.

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{W}\mathbf{x}^k - \gamma \mathbf{s}^k, \\ \mathbf{s}^{k+1} &= \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)\end{aligned}$$

- $\mathbf{x}^k$ is not consensual, i.e., $\mathbf{x}^k \neq \mathbf{1}\bar{\mathbf{x}}^k$.
- We need to bound the error caused by gradient disagreement.

We have:

$$f(\bar{\mathbf{x}}^k) - f(\mathbf{x}^*) \leq \langle \bar{\mathbf{s}}^k, \bar{\mathbf{x}}^k - \mathbf{x}^* \rangle - \frac{\mu}{4} \|\bar{\mathbf{x}}^k - \mathbf{x}^*\|^2 + \frac{L}{n} \|\Pi \mathbf{x}^k\|_F^2$$

Here, $\Pi \mathbf{x}^k = \mathbf{x}^k - \frac{1}{n} \mathbf{1} \mathbf{1}^\top \mathbf{x}^k$ measures the deviation from consensus.

Conclusion: To ensure a small upper bound, $\mathbf{x}^k$ must be close to consensus — this limits the allowable stepsize γ .

Accelerated Gradient Tracking: acc-DNGD-SC [Qu-Li, 2020]

Initialization: $\mathbf{z}^0 = \mathbf{x}^0$, $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$

$$\mathbf{z}^{k+1} = (1 - \alpha)\mathbf{W}\mathbf{z}^k + \alpha\mathbf{W}\mathbf{x}^k - \frac{\gamma}{\alpha}\mathbf{s}^k,$$

$$\mathbf{y}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma\mathbf{s}^k,$$

$$\mathbf{x}^{k+1} = \frac{\mathbf{y}^{k+1} + \alpha\mathbf{z}^{k+1}}{1 + \alpha},$$

$$\mathbf{s}^{k+1} = \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

Accelerated Gradient Tracking: acc-DNGD-SC [Qu-Li, 2020]

Initialization: $\mathbf{z}^0 = \mathbf{x}^0$, $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$

$$\mathbf{z}^{k+1} = (1 - \alpha)\mathbf{W}\mathbf{z}^k + \alpha\mathbf{W}\mathbf{x}^k - \frac{\gamma}{\alpha}\mathbf{s}^k,$$

$$\mathbf{y}^{k+1} = \mathbf{W}\mathbf{x}^k - \gamma\mathbf{s}^k,$$

$$\mathbf{x}^{k+1} = \frac{\mathbf{y}^{k+1} + \alpha\mathbf{z}^{k+1}}{1 + \alpha},$$

$$\mathbf{s}^{k+1} = \mathbf{W}\mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

- Based on the accelerated centralized algorithm [Nesterov '13: Scheme (2.2.6)], replacing $\nabla f(\mathbf{x})$ with $\mathbf{s}$.
- Complexity: $O\left(\frac{\kappa^{5/7}}{\theta^{3/2}} \log \frac{1}{\epsilon}\right)$

Accelerated Gradient Tracking: Acc-GT [Li-Lin, 2021]

Initialization: $\mathbf{y}^0 = \mathbf{z}^0 = \mathbf{x}^0$, $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \frac{\mu\gamma}{\alpha_k}} \left(\mathbf{W} \left(\frac{\mu\gamma}{\alpha_k} \mathbf{x}^k + \mathbf{z}^k \right) - \frac{\gamma}{\alpha_k} \mathbf{s}^k \right),$$

$$\mathbf{y}^{k+1} = (1 - \alpha_k) \mathbf{W} \mathbf{y}^k + \alpha_k \mathbf{z}^{k+1} = \mathbf{W} \mathbf{x}^k + \alpha_k (\mathbf{z}^{k+1} - \mathbf{W} \mathbf{z}^k),$$

$$\mathbf{x}^{k+1} = (1 - \alpha_{k+1}) \mathbf{y}^{k+1} + \alpha_{k+1} \mathbf{z}^{k+1},$$

$$\mathbf{s}^{k+1} = \mathbf{W} \mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

Accelerated Gradient Tracking: Acc-GT [Li-Lin, 2021]

Initialization: $\mathbf{y}^0 = \mathbf{z}^0 = \mathbf{x}^0$, $\mathbf{s}^0 = \nabla f(\mathbf{x}^0)$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \frac{\mu\gamma}{\alpha_k}} \left(\mathbf{W} \left(\frac{\mu\gamma}{\alpha_k} \mathbf{x}^k + \mathbf{z}^k \right) - \frac{\gamma}{\alpha_k} \mathbf{s}^k \right),$$

$$\mathbf{y}^{k+1} = (1 - \alpha_k) \mathbf{W} \mathbf{y}^k + \alpha_k \mathbf{z}^{k+1} = \mathbf{W} \mathbf{x}^k + \alpha_k (\mathbf{z}^{k+1} - \mathbf{W} \mathbf{z}^k),$$

$$\mathbf{x}^{k+1} = (1 - \alpha_{k+1}) \mathbf{y}^{k+1} + \alpha_{k+1} \mathbf{z}^{k+1},$$

$$\mathbf{s}^{k+1} = \mathbf{W} \mathbf{s}^k + \nabla f(\mathbf{x}^{k+1}) - \nabla f(\mathbf{x}^k)$$

- Based on the accelerated centralized algorithm from [Nesterov '13], adapted by replacing $\nabla f(\mathbf{x})$ with $\mathbf{s}$.
- When $\mu = 0$, this reduces to acc-DNGD.
- Complexity: $O\left(\frac{\sqrt{\kappa}}{\theta^{3/2}} \log \frac{1}{\epsilon}\right)$ for strongly convex objectives.

Optimal Complexity of Decentralized Optimization

Lower bounds for smooth and strongly convex problems [Scaman et al., 2017]:

- Gradient computation complexity:

$$O\left(\sqrt{\kappa} \log \frac{1}{\epsilon}\right)$$

- Communication complexity:

$$O\left(\sqrt{\frac{\kappa}{\theta}} \log \frac{1}{\epsilon}\right)$$

Optimal Complexity of Decentralized Optimization

Lower bounds for smooth and strongly convex problems [Scaman et al., 2017]:

- Gradient computation complexity:

$$O\left(\sqrt{\kappa} \log \frac{1}{\epsilon}\right)$$

- Communication complexity:

$$O\left(\sqrt{\frac{\kappa}{\theta}} \log \frac{1}{\epsilon}\right)$$

Definitions:

- Condition number of the function:

$$\kappa = \frac{L}{\mu}$$

- Spectral gap of the mixing matrix $\mathbf{W}$:

$$\theta = 1 - \max(|\lambda_2(\mathbf{W})|, |\lambda_n(\mathbf{W})|) = 1 - \max(\lambda_2(\mathbf{W}), -\lambda_n(\mathbf{W}))$$

Optimal Complexity of Decentralized Optimization

Lower bounds for smooth and strongly convex problems [Scaman et al., 2017]:

- Gradient computation complexity:

$$O\left(\sqrt{\kappa} \log \frac{1}{\epsilon}\right)$$

- Communication complexity:

$$O\left(\sqrt{\frac{\kappa}{\theta}} \log \frac{1}{\epsilon}\right)$$

Definitions:

- Condition number of the function:

$$\kappa = \frac{L}{\mu}$$

- Spectral gap of the mixing matrix $\mathbf{W}$:

$$\theta = 1 - \max(|\lambda_2(\mathbf{W})|, |\lambda_n(\mathbf{W})|) = 1 - \max(\lambda_2(\mathbf{W}), -\lambda_n(\mathbf{W}))$$

Implications:

- To remove dependence on θ from gradient complexity, **additional communication rounds** are typically needed.

Improving Dependence on θ

Let the eigenvalues of the mixing matrix $\mathbf{W}$ satisfy:

$$1 = \lambda_1 > \lambda_2 \geq \dots \geq \lambda_n.$$

- **Finite-step consensus (exact eigenvalues known):** Construct a polynomial p of degree n such that

$$p(1) = 1, \quad p(\lambda_i) = 0 \quad \forall i = 2, \dots, n.$$

Then applying $p(\mathbf{W})$ achieves exact consensus in n steps.

Improving Dependence on θ

Let the eigenvalues of the mixing matrix $\mathbf{W}$ satisfy:

$$1 = \lambda_1 > \lambda_2 \geq \dots \geq \lambda_n.$$

- **Finite-step consensus (exact eigenvalues known):** Construct a polynomial p of degree n such that

$$p(1) = 1, \quad p(\lambda_i) = 0 \quad \forall i = 2, \dots, n.$$

Then applying $p(\mathbf{W})$ achieves exact consensus in n steps.

- **Multiple communications:**

Replace $\mathbf{W}$ with $\mathbf{W}^K$. This is equivalent to applying the monomial polynomial $p(\lambda) = \lambda^K$. Then:

$$\max(|\lambda_2|, |\lambda_n|)^K \leq (1 - \theta)^{1/\theta} < \frac{1}{e}, \quad \text{if } K > \frac{1}{\theta}.$$

This amplifies the spectral gap and improves mixing. However, if θ is not known exactly, conservative lower bounds may lead to overestimated K (more communication rounds).

Improving Dependence on θ

Let the eigenvalues of the mixing matrix $\mathbf{W}$ satisfy:

$$1 = \lambda_1 > \lambda_2 \geq \dots \geq \lambda_n.$$

- **Finite-step consensus (exact eigenvalues known):** Construct a polynomial p of degree n such that

$$p(1) = 1, \quad p(\lambda_i) = 0 \quad \forall i = 2, \dots, n.$$

Then applying $p(\mathbf{W})$ achieves exact consensus in n steps.

- **Multiple communications:**

Replace $\mathbf{W}$ with $\mathbf{W}^K$. This is equivalent to applying the monomial polynomial $p(\lambda) = \lambda^K$. Then:

$$\max(|\lambda_2|, |\lambda_n|)^K \leq (1 - \theta)^{1/\theta} < \frac{1}{e}, \quad \text{if } K > \frac{1}{\theta}.$$

This amplifies the spectral gap and improves mixing. However, if θ is not known exactly, conservative lower bounds may lead to overestimated K (more communication rounds).

Goal: Construct a better polynomial p such that

$$p(1) = 1, \quad |p(\lambda)| \ll 1 \quad \text{for all } \lambda \in [\ell, u] \supset [\lambda_n, \lambda_2].$$

Improving Dependence on θ

Let the eigenvalues of the mixing matrix $\mathbf{W}$ satisfy:

$$1 = \lambda_1 > \lambda_2 \geq \dots \geq \lambda_n.$$

- **Finite-step consensus (exact eigenvalues known):** Construct a polynomial p of degree n such that

$$p(1) = 1, \quad p(\lambda_i) = 0 \quad \forall i = 2, \dots, n.$$

Then applying $p(\mathbf{W})$ achieves exact consensus in n steps.

- **Multiple communications:**

Replace $\mathbf{W}$ with $\mathbf{W}^K$. This is equivalent to applying the monomial polynomial $p(\lambda) = \lambda^K$. Then:

$$\max(|\lambda_2|, |\lambda_n|)^K \leq (1 - \theta)^{1/\theta} < \frac{1}{e}, \quad \text{if } K > \frac{1}{\theta}.$$

This amplifies the spectral gap and improves mixing. However, if θ is not known exactly, conservative lower bounds may lead to overestimated K (more communication rounds).

Goal: Construct a better polynomial p such that

$$p(1) = 1, \quad |p(\lambda)| \ll 1 \quad \text{for all } \lambda \in [\ell, u] \supset [\lambda_n, \lambda_2].$$

If λ_2 and λ_n are unknown, we must estimate a tight spectral interval $[\ell, u]$.

Improving Dependence on θ : Chebyshev Polynomial Filtering

Chebyshev polynomials $T_k(x)$ satisfy:

$$|T_k(x)| \leq 1, \quad \forall x \in [-1, 1].$$

Improving Dependence on θ : Chebyshev Polynomial Filtering

Chebyshev polynomials $T_k(x)$ satisfy:

$$|T_k(x)| \leq 1, \quad \forall x \in [-1, 1].$$

- **Step 1: Rescale the spectrum.**

Suppose eigenvalues $\lambda_i \in [\ell, u] \subset (-1, 1)$ for $i = 2, \dots, n$. Define the rescaled variable:

$$\hat{\lambda} = \frac{2}{u - \ell} \left(\lambda - \frac{\ell + u}{2} \right), \quad \text{so } \hat{\lambda} \in [-1, 1].$$

The consensus eigenvalue $\lambda_1 = 1$ is mapped to:

$$\hat{\lambda}_1 = \frac{2 - \ell - u}{u - \ell} > 1.$$

Improving Dependence on θ : Chebyshev Polynomial Filtering

Chebyshev polynomials $T_k(x)$ satisfy:

$$|T_k(x)| \leq 1, \quad \forall x \in [-1, 1].$$

- **Step 1: Rescale the spectrum.**

Suppose eigenvalues $\lambda_i \in [\ell, u] \subset (-1, 1)$ for $i = 2, \dots, n$. Define the rescaled variable:

$$\hat{\lambda} = \frac{2}{u - \ell} \left(\lambda - \frac{\ell + u}{2} \right), \quad \text{so } \hat{\lambda} \in [-1, 1].$$

The consensus eigenvalue $\lambda_1 = 1$ is mapped to:

$$\hat{\lambda}_1 = \frac{2 - \ell - u}{u - \ell} > 1.$$

- **Step 2: Construct the Chebyshev polynomial.**

Let $\hat{p}_k(\hat{\lambda}) = T_k(\hat{\lambda})$ be the Chebyshev polynomial of degree k .

Improving Dependence on θ : Chebyshev Polynomial Filtering

Chebyshev polynomials $T_k(x)$ satisfy:

$$|T_k(x)| \leq 1, \quad \forall x \in [-1, 1].$$

- **Step 1: Rescale the spectrum.**

Suppose eigenvalues $\lambda_i \in [\ell, u] \subset (-1, 1)$ for $i = 2, \dots, n$. Define the rescaled variable:

$$\hat{\lambda} = \frac{2}{u - \ell} \left(\lambda - \frac{\ell + u}{2} \right), \quad \text{so } \hat{\lambda} \in [-1, 1].$$

The consensus eigenvalue $\lambda_1 = 1$ is mapped to:

$$\hat{\lambda}_1 = \frac{2 - \ell - u}{u - \ell} > 1.$$

- **Step 2: Construct the Chebyshev polynomial.**

Let $\hat{p}_k(\hat{\lambda}) = T_k(\hat{\lambda})$ be the Chebyshev polynomial of degree k .

- **Step 3: Normalize to satisfy $p(1) = 1$.**

$$p(\lambda) = \frac{T_k(\hat{\lambda})}{T_k(\hat{\lambda}_1)}.$$

Improving Dependence on θ : Chebyshev Polynomial Filtering

Chebyshev polynomials $T_k(x)$ satisfy:

$$|T_k(x)| \leq 1, \quad \forall x \in [-1, 1].$$

- **Step 1: Rescale the spectrum.**

Suppose eigenvalues $\lambda_i \in [\ell, u] \subset (-1, 1)$ for $i = 2, \dots, n$. Define the rescaled variable:

$$\hat{\lambda} = \frac{2}{u - \ell} \left(\lambda - \frac{\ell + u}{2} \right), \quad \text{so } \hat{\lambda} \in [-1, 1].$$

The consensus eigenvalue $\lambda_1 = 1$ is mapped to:

$$\hat{\lambda}_1 = \frac{2 - \ell - u}{u - \ell} > 1.$$

- **Step 2: Construct the Chebyshev polynomial.**

Let $\hat{p}_k(\hat{\lambda}) = T_k(\hat{\lambda})$ be the Chebyshev polynomial of degree k .

- **Step 3: Normalize to satisfy $p(1) = 1$.**

$$p(\lambda) = \frac{T_k(\hat{\lambda})}{T_k(\hat{\lambda}_1)}.$$

- **Result:** $p(1) = 1$ and $|p(\lambda_i)| \leq \frac{1}{|T_k(\hat{\lambda}_1)|}$ for $i \geq 2$.

So, a larger $\hat{\lambda}_1$ gives better suppression of non-consensus components.

Chebyshev Acceleration [Arioli-Scott, 2014; Scaman et al., 2017]

Chebyshev Polynomials:

$$\begin{aligned}T_0(x) &= 1, & T_1(x) &= x, \\T_{k+1}(x) &= 2xT_k(x) - T_{k-1}(x)\end{aligned}$$

Chebyshev Acceleration [Arioli-Scott, 2014; Scaman et al., 2017]

Chebyshev Polynomials:

$$\begin{aligned}T_0(x) &= 1, & T_1(x) &= x, \\T_{k+1}(x) &= 2xT_k(x) - T_{k-1}(x)\end{aligned}$$

Rescaling:

$$\widehat{\mathbf{W}} = \frac{2}{u - \ell}(\mathbf{W} - \frac{u + \ell}{2}\mathbf{I}), \quad \hat{\lambda}_1 := \frac{2 - \ell - u}{u - \ell}$$

Chebyshev Acceleration [Arioli-Scott, 2014; Scaman et al., 2017]

Chebyshev Polynomials:

$$T_0(x) = 1, \quad T_1(x) = x, \\ T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x)$$

Rescaling:

$$\widehat{\mathbf{W}} = \frac{2}{u - \ell}(\mathbf{W} - \frac{u + \ell}{2}\mathbf{I}), \quad \hat{\lambda}_1 := \frac{2 - \ell - u}{u - \ell}$$

Chebyshev Acceleration Procedure:

- Initialize:

$$a_0 = 1, \quad a_1 = \hat{\lambda}_1, \quad \mathbf{x}^0 = \mathbf{x}, \quad \mathbf{x}^1 = \widehat{\mathbf{W}}\mathbf{x}$$

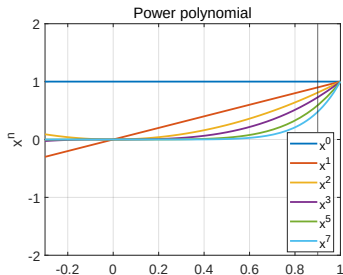
- For $k = 1, \dots, K - 1$, iterate:

$$a_{k+1} = 2\hat{\lambda}_1 a_k - a_{k-1} \\ \mathbf{x}^{k+1} = 2\widehat{\mathbf{W}}\mathbf{x}^k - \mathbf{x}^{k-1}$$

- Return the accelerated result:

$$\boxed{\mathbf{x}_{\text{final}} = \frac{\mathbf{x}^K}{a_K}}$$

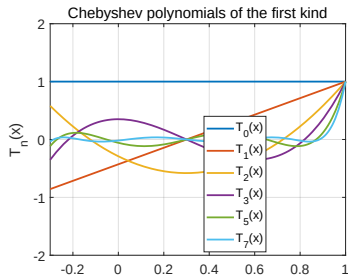
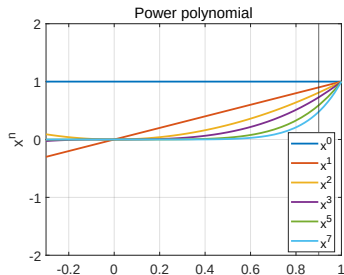
Polynomial Decay: Power vs Chebyshev



- **Power Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| = \max(|\ell|, |u|)^K$$

Polynomial Decay: Power vs Chebyshev



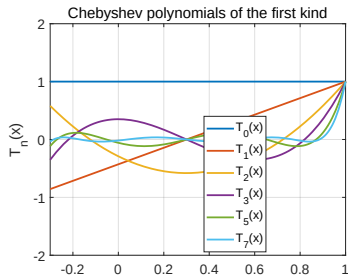
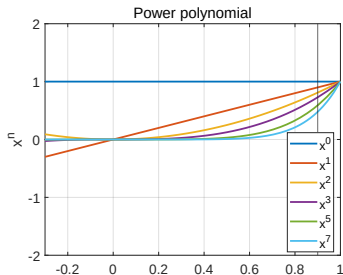
- Power Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| = \max(|\ell|, |u|)^K$$

- Chebyshev Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| = \frac{2}{\left(\frac{2-\ell-u}{u-\ell} - \sqrt{\frac{(2-\ell-u)^2}{(u-\ell)^2} - 1} \right)^K + \left(\frac{2-\ell-u}{u-\ell} + \sqrt{\frac{(2-\ell-u)^2}{(u-\ell)^2} - 1} \right)^K}$$

Polynomial Decay: Power vs Chebyshev



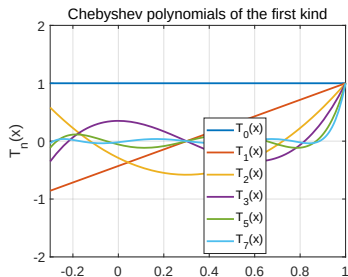
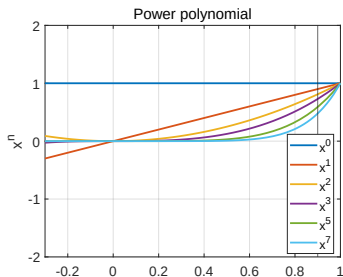
- Power Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| = \max(|\ell|, |u|)^K$$

- Chebyshev Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| \leq 2 \left(\frac{\sqrt{1-\ell} - \sqrt{1-u}}{\sqrt{1-\ell} + \sqrt{1-u}} \right)^K \leq 2 \left(1 - \frac{2\sqrt{1-u}}{\sqrt{1-\ell} + \sqrt{1-u}} \right)^K$$

Polynomial Decay: Power vs Chebyshev



- **Power Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| = \max(|\ell|, |u|)^K$$

- **Chebyshev Method:**

$$\max_{x \in [\ell, u]} |p_K(x)| \leq 2 \left(\frac{\sqrt{1-\ell} - \sqrt{1-u}}{\sqrt{1-\ell} + \sqrt{1-u}} \right)^K \leq 2 \left(1 - \frac{2\sqrt{1-u}}{\sqrt{1-\ell} + \sqrt{1-u}} \right)^K$$

Conclusion: Chebyshev polynomials provide faster decay over the interval $[\ell, u]$ compared to power iterations.

Chebyshev Acceleration Removes θ from Computation Complexity

Let $K \geq \frac{1}{\sqrt{1-u}} + \frac{1}{\sqrt{2}} \geq \frac{\sqrt{1-l} + \sqrt{1-u}}{\sqrt{2(1-u)}}$, then the Chebyshev polynomial satisfies:

$$\max_{x \in [l, u]} |p_K(x)| \leq \frac{2}{e\sqrt{2}} < 0.5$$

Chebyshev Acceleration Removes θ from Computation Complexity

Let $K \geq \frac{1}{\sqrt{1-u}} + \frac{1}{\sqrt{2}} \geq \frac{\sqrt{1-l} + \sqrt{1-u}}{\sqrt{2(1-u)}}$, then the Chebyshev polynomial satisfies:

$$\max_{x \in [l, u]} |p_K(x)| \leq \frac{2}{e\sqrt{2}} < 0.5$$

- After $K = O\left(\frac{1}{\sqrt{\theta}}\right)$ communication rounds, the **new effective spectral gap** becomes at least 0.5 — a constant independent of $\mathbf{W}$.

Chebyshev Acceleration Removes θ from Computation Complexity

Let $K \geq \frac{1}{\sqrt{1-u}} + \frac{1}{\sqrt{2}} \geq \frac{\sqrt{1-l} + \sqrt{1-u}}{\sqrt{2(1-u)}}$, then the Chebyshev polynomial satisfies:

$$\max_{x \in [l, u]} |p_K(x)| \leq \frac{2}{e\sqrt{2}} < 0.5$$

- After $K = O\left(\frac{1}{\sqrt{\theta}}\right)$ communication rounds, the **new effective spectral gap** becomes at least 0.5 — a constant independent of $\mathbf{W}$.
- This leads to significantly improved overall complexity:
 - GT (Gradient Tracking):

$$\text{Computation: } O\left(\kappa \log \frac{1}{\epsilon}\right), \quad \text{Communication: } O\left(\frac{\kappa}{\sqrt{\theta}} \log \frac{1}{\epsilon}\right)$$

- acc-DNGD-SC (Accelerated DNGD with Strong Convexity):

$$O\left(\frac{\kappa^{5/7}}{\sqrt{\theta}} \log \frac{1}{\epsilon}\right) \text{ communication}$$

- Acc-GT (Accelerated Gradient Tracking):

$$\textbf{Optimal: } O\left(\sqrt{\frac{\kappa}{\theta}} \log \frac{1}{\epsilon}\right) \text{ communication}$$

Limitations of Inner-Loop Chebyshev Acceleration

In practice, inner-loop methods can be inefficient.

- As observed in [Li-Lin, 2021]:

“Acc-GT-C requires more communication rounds than Acc-GT to reach the same objective function accuracy, even though it has lower theoretical communication complexity. Thus, Acc-GT-C is primarily of theoretical interest and not recommended in practice.”

Limitations of Inner-Loop Chebyshev Acceleration

In practice, inner-loop methods can be inefficient.

- As observed in [Li-Lin, 2021]:

“Acc-GT-C requires more communication rounds than Acc-GT to reach the same objective function accuracy, even though it has lower theoretical communication complexity. Thus, Acc-GT-C is primarily of theoretical interest and not recommended in practice.”

Are there alternative approaches?

Connect decentralized optimization to stochastic variance reduction techniques.

Aim to achieve optimal *computation complexity* without dependence on the network spectral gap θ .

SS-GT: “Snapshot” Gradient Tracking

- Use acceleration to achieve computation complexity of $O(\sqrt{\kappa})$ (e.g., Acc-GT). $\Rightarrow$ Inspired by Katyusha

Gradient Descent and Nesterov Acceleration

$$\min_{\mathbf{x}} \bar{f}(\mathbf{x}) := \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

- **Gradient Descent (GD):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla \bar{f}(\mathbf{x}^k)$$

- $O\left(\frac{nL}{\epsilon}\right)$ for general convex ($\mu = 0$).
- $O(n\kappa \log \frac{1}{\epsilon})$ for strongly convex.

(Counts number of ∇f_i evaluations.)

Gradient Descent and Nesterov Acceleration

$$\min_{\mathbf{x}} \bar{f}(\mathbf{x}) := \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

- **Gradient Descent (GD):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla \bar{f}(\mathbf{x}^k)$$

- $O\left(\frac{nL}{\epsilon}\right)$ for general convex ($\mu = 0$).
- $O(n\kappa \log \frac{1}{\epsilon})$ for strongly convex.

(Counts number of ∇f_i evaluations.)

- **Nesterov Acceleration:**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla \bar{f}(\mathbf{x}^k) + \alpha_k (\mathbf{x}^k - \mathbf{x}^{k-1})$$

- $O\left(\frac{nL}{\sqrt{\epsilon}}\right)$ for general convex.
- $O(n\sqrt{\kappa} \log \frac{1}{\epsilon})$ for strongly convex.

SGD and Acceleration

$$\min_{\mathbf{x}} \bar{f}(\mathbf{x}) := \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

- **Stochastic Gradient Descent (SGD):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f_{i_k}(\mathbf{x}^k)$$

$O\left(\frac{1}{\epsilon}\right)$ for strongly convex problems.

SGD and Acceleration

$$\min_{\mathbf{x}} \bar{f}(\mathbf{x}) := \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

- **Stochastic Gradient Descent (SGD):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f_{i_k}(\mathbf{x}^k)$$

$O\left(\frac{1}{\epsilon}\right)$ for strongly convex problems.

- **Variance-Reduced SGD (e.g., SVRG):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \left[\nabla f_{i_k}(\mathbf{x}^k) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \right]$$

$O\left((n + \kappa) \log \frac{1}{\epsilon}\right)$ for strongly convex problems.

SGD and Acceleration

$$\min_{\mathbf{x}} \bar{f}(\mathbf{x}) := \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

- **Stochastic Gradient Descent (SGD):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f_{i_k}(\mathbf{x}^k)$$

$O\left(\frac{1}{\epsilon}\right)$ for strongly convex problems.

- **Variance-Reduced SGD (e.g., SVRG):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \left[\nabla f_{i_k}(\mathbf{x}^k) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \right]$$

$O\left((n + \kappa) \log \frac{1}{\epsilon}\right)$ for strongly convex problems.

- **Acceleration:**

- APPA [Frostig et al., 2015]
- Catalyst [Lin et al., 2015]

$O\left((n + \sqrt{n\kappa}) \log \kappa \log \frac{1}{\epsilon}\right)$ for strongly convex problems.

(The extra $\log \kappa$ arises from solving subproblems via inner loops.)

Katyusha

$$\mathbf{x}^{k+1} = \tau_1 \mathbf{z}^k + \tau_2 \tilde{\mathbf{x}} + (1 - \tau_1 - \tau_2) \mathbf{y}^k$$

$$\mathbf{g}^{k+1} = \nabla f_{i_k}(\mathbf{x}^{k+1}) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \quad // \text{ variance reduction step}$$

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \alpha \mathbf{g}^{k+1} \quad // \text{ SGD-like step}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^{k+1} + \tau_1 (\mathbf{z}^{k+1} - \mathbf{z}^k) = \mathbf{x}^{k+1} - \tau_1 \alpha \mathbf{g}^{k+1}$$

Katyusha

$$\mathbf{x}^{k+1} = \tau_1 \mathbf{z}^k + \tau_2 \tilde{\mathbf{x}} + (1 - \tau_1 - \tau_2) \mathbf{y}^k$$

$$\mathbf{g}^{k+1} = \nabla f_{i_k}(\mathbf{x}^{k+1}) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \quad // \text{ variance reduction step}$$

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \alpha \mathbf{g}^{k+1} \quad // \text{ SGD-like step}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^{k+1} + \tau_1 (\mathbf{z}^{k+1} - \mathbf{z}^k) = \mathbf{x}^{k+1} - \tau_1 \alpha \mathbf{g}^{k+1}$$

- When $\tau_1 = \tau_2 = 0$: no acceleration — reduces to variance-reduced SGD.

Katyusha

$$\mathbf{x}^{k+1} = \tau_1 \mathbf{z}^k + \tau_2 \tilde{\mathbf{x}} + (1 - \tau_1 - \tau_2) \mathbf{y}^k$$

$$\mathbf{g}^{k+1} = \nabla f_{i_k}(\mathbf{x}^{k+1}) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \quad // \text{ variance reduction step}$$

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \alpha \mathbf{g}^{k+1} \quad // \text{ SGD-like step}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^{k+1} + \tau_1 (\mathbf{z}^{k+1} - \mathbf{z}^k) = \mathbf{x}^{k+1} - \tau_1 \alpha \mathbf{g}^{k+1}$$

- When $\tau_1 = \tau_2 = 0$: no acceleration — reduces to variance-reduced SGD.
- When $\tau_2 = 0$ and $\mathbf{x}^0 = \mathbf{y}^0 = \mathbf{z}^0$: recovers linear coupling (acceleration) with SGD.

$$\mathbf{y}^1 = \mathbf{x}^1 - \tau_1 \alpha \mathbf{g}^1 = \mathbf{x}^0 - \tau_1 \alpha \mathbf{g}^1$$

$$\begin{aligned} \mathbf{y}^2 &= \tau_1 (\mathbf{x}^0 - \alpha \mathbf{g}^1) + (1 - \tau_1) (\mathbf{x}^0 - \tau_1 \alpha \mathbf{g}^1) - \tau_1 \alpha \mathbf{g}^2 \\ &= \mathbf{x}^0 - (1 + 1 - \tau_1) \tau_1 \alpha \mathbf{g}^1 - \tau_1 \alpha \mathbf{g}^2 \end{aligned}$$

$$\begin{aligned} \mathbf{y}^3 &= \tau_1 (\mathbf{x}^0 - \alpha \mathbf{g}^1 - \alpha \mathbf{g}^2) + (1 - \tau_1) (\mathbf{x}^0 - (1 + 1 - \tau_1) \tau_1 \alpha \mathbf{g}^1 - \tau_1 \alpha \mathbf{g}^2) - \tau_1 \alpha \mathbf{g}^3 \\ &= \mathbf{x}^0 - (1 + 1 - \tau_1 + (1 - \tau_1)^2) \tau_1 \alpha \mathbf{g}^1 - (1 + 1 - \tau_1) \tau_1 \alpha \mathbf{g}^2 - \tau_1 \alpha \mathbf{g}^3 \end{aligned}$$

Katyusha

$$\mathbf{x}^{k+1} = \tau_1 \mathbf{z}^k + \tau_2 \tilde{\mathbf{x}} + (1 - \tau_1 - \tau_2) \mathbf{y}^k$$

$$\mathbf{g}^{k+1} = \nabla f_{i_k}(\mathbf{x}^{k+1}) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \quad // \text{ variance reduction step}$$

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \alpha \mathbf{g}^{k+1} \quad // \text{ SGD-like step}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^{k+1} + \tau_1 (\mathbf{z}^{k+1} - \mathbf{z}^k) = \mathbf{x}^{k+1} - \tau_1 \alpha \mathbf{g}^{k+1}$$

- When $\tau_1 = \tau_2 = 0$: no acceleration — reduces to variance-reduced SGD.
- When $\tau_2 = 0$ and $\mathbf{x}^0 = \mathbf{y}^0 = \mathbf{z}^0$: recovers linear coupling (acceleration) with SGD.
- The term $\tau_2(\tilde{\mathbf{x}} - \mathbf{y}^k)$ acts as **negative momentum**: it pulls iterates back toward the stable snapshot $\tilde{\mathbf{x}}$, improving stability and reducing variance.

Katyusha

$$\mathbf{x}^{k+1} = \tau_1 \mathbf{z}^k + \tau_2 \tilde{\mathbf{x}} + (1 - \tau_1 - \tau_2) \mathbf{y}^k$$

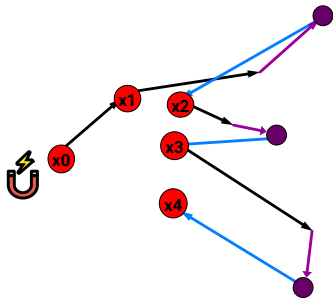
$$\mathbf{g}^{k+1} = \nabla f_{i_k}(\mathbf{x}^{k+1}) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla \bar{f}(\tilde{\mathbf{x}}) \quad // \text{ variance reduction step}$$

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \alpha \mathbf{g}^{k+1} \quad // \text{ SGD-like step}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^{k+1} + \tau_1 (\mathbf{z}^{k+1} - \mathbf{z}^k) = \mathbf{x}^{k+1} - \tau_1 \alpha \mathbf{g}^{k+1}$$

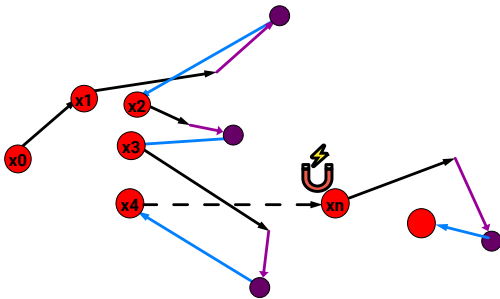
- When $\tau_1 = \tau_2 = 0$: no acceleration — reduces to variance-reduced SGD.
- When $\tau_2 = 0$ and $\mathbf{x}^0 = \mathbf{y}^0 = \mathbf{z}^0$: recovers linear coupling (acceleration) with SGD.
- The term $\tau_2(\tilde{\mathbf{x}} - \mathbf{y}^k)$ acts as **negative momentum**: it pulls iterates back toward the stable snapshot $\tilde{\mathbf{x}}$, improving stability and reducing variance.
- Suggested hyperparameters:
 - $\tau_2 = 0.5$
 - $\tau_1 = \min \left\{ \sqrt{\frac{n}{\kappa}}, 0.5 \right\}$

Katyusha



Step 1: Variance reduction with snapshot $\tilde{x}$

Katyusha



Step 2: Update $\tilde{\mathbf{x}}$

SS-GT: “Snapshot” Gradient Tracking

- Use acceleration to achieve computation complexity of $O(\sqrt{\kappa})$ (e.g., Acc-GT). $\Rightarrow$ Inspired by Katyusha

SS-GT: “Snapshot” Gradient Tracking

- Use acceleration to achieve computation complexity of $O(\sqrt{\kappa})$ (e.g., Acc-GT). $\Rightarrow$ Inspired by Katyusha
- Eliminate dependence on θ in gradient computation complexity **without** relying on inner-loop Chebyshev acceleration. $\Rightarrow$ Use Bernoulli sampling strategy

SS-GT: Snapshot GT

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{q}^k \quad (\text{similar to Katyusha})$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} (\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k)\mathbf{q}^k \quad (\text{snapshot updates when } \xi^k = 1)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}))$$

Random variables: $\xi^k \sim \text{Bernoulli}(p)$, .

SS-GT: Snapshot GT

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{q}^k \quad (\text{similar to Katyusha})$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} (\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k \quad (\text{snapshot updates when } \xi^k = 1)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \xi^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

Random variables: $\xi^k \sim \text{Bernoulli}(p)$, .

- $\bar{\mathbf{g}}^k = \frac{1}{n} \mathbf{1}^\top \nabla f(\mathbf{q}^k)$ is the average gradient.

SS-GT: Snapshot GT

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k \quad (\text{similar to Katyusha})$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} (\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k \quad (\text{snapshot updates when } \xi^k = 1)$$

$$\mathbf{u}^{k+1} = \mathbf{W} (\xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{u}^k)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \xi^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

Random variables: $\xi^k \sim \text{Bernoulli}(p)$, .

- Using the snapshot point $\mathbf{u}^k$ leads to a smaller consensus error compared to $\mathbf{q}^k$.

SS-GT: Snapshot GT

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k \quad (\text{similar to Katyusha})$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} (\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k \quad (\text{snapshot updates when } \xi^k = 1)$$

$$\mathbf{u}^{k+1} = \mathbf{W} (\xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{u}^k)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

Random variables: $\xi^k \sim \text{Bernoulli}(p)$, $\zeta^k \sim \frac{1}{q} \text{Bernoulli}(q)$ so that $\mathbb{E}[\zeta^k] = 1$.

- Using the snapshot point $\mathbf{u}^k$ leads to a smaller consensus error compared to $\mathbf{q}^k$.
- $\mathbf{1}^\top \mathbf{g}^k = \mathbf{1}^\top \nabla f(\mathbf{q}^k) \neq \mathbf{1}^\top \nabla f(\mathbf{x}^k)$, but the correction term $\mathbb{E}[\zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k))] + \mathbf{g}^k$ restores the average while reducing the consensus error.

SS-GT: Snapshot GT

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k \quad (\text{similar to Katyusha})$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} \left(\mathbf{z}^k + \beta\mathbf{x}^k - \eta \left(\zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) + \mathbf{g}^k \right) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k \quad (\text{snapshot updates when } \xi^k = 1)$$

$$\mathbf{u}^{k+1} = \mathbf{W} \left(\xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{u}^k \right)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

Random variables: $\xi^k \sim \text{Bernoulli}(p)$, $\zeta^k \sim \frac{1}{q} \text{Bernoulli}(q)$ so that $\mathbb{E}[\zeta^k] = 1$.

- Using the snapshot point $\mathbf{u}^k$ leads to a smaller consensus error compared to $\mathbf{q}^k$.
- $\mathbf{1}^\top \mathbf{g}^k = \mathbf{1}^\top \nabla f(\mathbf{q}^k) \neq \mathbf{1}^\top \nabla f(\mathbf{x}^k)$, but the correction term $\mathbb{E}[\zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k))] + \mathbf{g}^k$ restores the average while reducing the consensus error.

SS-GT: Snapshot GT

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k \quad (\text{similar to Katyusha})$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} \left(\mathbf{z}^k + \beta\mathbf{x}^k - \eta \left(\zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) + \mathbf{g}^k \right) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k \quad (\text{snapshot updates when } \xi^k = 1)$$

$$\mathbf{u}^{k+1} = \mathbf{W} \left(\xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{u}^k \right)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

Random variables: $\xi^k \sim \text{Bernoulli}(p)$, $\zeta^k \sim \frac{1}{q} \text{Bernoulli}(q)$ so that $\mathbb{E}[\zeta^k] = 1$.

- Using the snapshot point $\mathbf{u}^k$ leads to a smaller consensus error compared to $\mathbf{q}^k$.
- $\mathbf{1}^\top \mathbf{g}^k = \mathbf{1}^\top \nabla f(\mathbf{q}^k) \neq \mathbf{1}^\top \nabla f(\mathbf{x}^k)$, but the correction term $\mathbb{E}[\zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k))] + \mathbf{g}^k$ restores the average while reducing the consensus error.
- **SS-GT Complexity:** Computation: $O(\sqrt{\kappa} \log \frac{1}{\epsilon})$ Communication: $O\left(\frac{\sqrt{\kappa}}{\theta} \log \frac{1}{\epsilon}\right)$
Parameters: $\tau = \frac{1}{2}$, $\alpha, \gamma = O\left(\frac{1}{\sqrt{\kappa}}\right)$, $p = q = O(\theta)$, $\eta = O\left(\frac{\theta}{\sqrt{\mu L}}\right)$, $\beta = O\left(\frac{\theta}{\sqrt{\kappa}}\right)$

SS-GT: Implementation

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\mathbf{z}^{k+1} = \frac{1}{1 + \beta} \mathbf{W} \left(\mathbf{z}^k + \beta\mathbf{x}^k - \eta \left(\zeta^k (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) + \mathbf{g}^k \right) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k)$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k$$

$$\mathbf{u}^{k+1} = \mathbf{W} \left(\xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{u}^k \right)$$

$$\mathbf{g}^{k+1} = \mathbf{W} \mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

SS-GT: Implementation

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\mathbf{z}^{k+1} = \begin{cases} \frac{1}{1+\beta} \mathbf{W} \left(\mathbf{z}^k + \beta\mathbf{x}^k - \frac{\eta}{q} (\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) - \eta\mathbf{g}^k \right), & \text{if } \zeta^k = \frac{1}{q} \\ \frac{1}{1+\beta} \mathbf{W} \left(\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k \right), & \text{else} \end{cases}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k)$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k$$

$$\mathbf{u}^{k+1} = \mathbf{W} \left(\xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{u}^k \right)$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

SS-GT: Implementation

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\mathbf{z}^{k+1} = \begin{cases} \frac{1}{1+\beta}\mathbf{W}\left(\mathbf{z}^k + \beta\mathbf{x}^k - \frac{\eta}{q}(\nabla f(\mathbf{x}^k) - \nabla f(\mathbf{q}^k)) - \eta\mathbf{g}^k\right), & \text{if } \zeta^k = \frac{1}{q} \\ \frac{1}{1+\beta}\mathbf{W}\left(\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k\right), & \text{else} \end{cases}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k)$$

$$\mathbf{q}^{k+1} = \begin{cases} \mathbf{x}^k, & \text{if } \xi^k = 1 \\ \mathbf{q}^k, & \text{else} \end{cases}$$

$$\mathbf{u}^{k+1} = \begin{cases} \mathbf{W}\mathbf{x}^k, & \text{if } \xi^k = 1 \\ \mathbf{W}\mathbf{u}^k, & \text{else} \end{cases}$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \nabla f(\mathbf{q}^{k+1}) - \nabla f(\mathbf{q}^k)$$

SS-GT: Implementation

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\mathbf{z}^{k+1} = \begin{cases} \frac{1}{1+\beta}\mathbf{W}(\mathbf{z}^k + \beta\mathbf{x}^k - \frac{\eta}{q}(\nabla f(\mathbf{x}^k) - \mathbf{m}^k) - \eta\mathbf{g}^k), & \text{if } \zeta^k = \frac{1}{q} \\ \frac{1}{1+\beta}\mathbf{W}(\mathbf{z}^k + \beta\mathbf{x}^k - \eta\mathbf{g}^k), & \text{else} \end{cases}$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k)$$

$$\mathbf{m}^{k+1} = \begin{cases} \nabla f(\mathbf{x}^k), & \text{if } \xi^k = 1 \\ \mathbf{m}^k, & \text{else} \end{cases}$$

$$\mathbf{u}^{k+1} = \begin{cases} \mathbf{W}\mathbf{x}^k, & \text{if } \xi^k = 1 \\ \mathbf{W}\mathbf{u}^k, & \text{else} \end{cases}$$

$$\mathbf{g}^{k+1} = \mathbf{W}\mathbf{g}^k + \mathbf{m}^{k+1} - \mathbf{m}^k$$

To reduce gradient computation, let $\mathbf{m}^k = \nabla f(\mathbf{q}^k)$.

Chebyshev Polynomial Recall

Standard Chebyshev recurrence:

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

It holds that $\max_{x \in [-1, 1]} T_n^2(x) = 1$. Even if we restrict to $x \in [0, 1 - \theta]$, the maximum still approaches 1 for large n .

In Chebyshev acceleration (inner-loop), both input and output must be rescaled.

Chebyshev Polynomial Recall

Standard Chebyshev recurrence:

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

It holds that $\max_{x \in [-1, 1]} T_n^2(x) = 1$. Even if we restrict to $x \in [0, 1 - \theta]$, the maximum still approaches 1 for large n .

In Chebyshev acceleration (inner-loop), both input and output must be rescaled.

Alternative recurrence from Liu–Morse (2011):

$$T_0(x) = 1, \quad T_1(x) = 1, \quad T_{n+1}(x) = (1 + \tilde{\eta})xT_n(x) - \tilde{\eta}T_{n-1}(x)$$

- When $\tilde{\eta} = 1$, this is similar to (but not exactly) the standard Chebyshev recurrence.
- If $1 > \tilde{\eta} \geq \frac{1}{1 + \sqrt{1 - (1 - \theta)^2}} \approx (1 - \sqrt{\theta/2})^2$, then

$$\sup_{k \geq 0} \sup_{x \in [0, 1 - \theta]} \frac{T_k^2(x)}{\tilde{\eta}^k} \leq c'_5 = 7.$$

That is, $\sup_{x \in [0, 1 - \theta]} T_k^2(x) \leq c'_5 \tilde{\eta}^k = c'_5 (\sqrt{\tilde{\eta}})^{2k}$, which decays exponentially.

How to use it?

Define the following matrices:

$$\widetilde{\Pi} = \begin{bmatrix} \Pi & \mathbf{0} \\ \mathbf{0} & \Pi \end{bmatrix}, \quad \mathbf{A}_{\#} = \begin{bmatrix} \mathbf{A} \\ \mathbf{A} \end{bmatrix}, \quad \widetilde{\mathbf{W}} = \begin{bmatrix} (1 + \tilde{\eta}_w) \mathbf{W} & -\tilde{\eta}_w \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}$$

Here, define the parameters:

$$\tilde{\eta}_w = \frac{1}{1 + \sqrt{1 - (1 - \theta)^2}}, \quad \tilde{\rho}_w = \sqrt{\tilde{\eta}_w}, \quad \tilde{\theta}_w = 1 - \tilde{\rho}_w = O(\sqrt{\theta})$$

Then we have the bound:

$$\|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \mathbf{A}_{\#}\|^2 = \|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \widetilde{\Pi} \mathbf{A}_{\#}\|^2 \leq 2c'_5 \tilde{\eta}_w^k \|\Pi \mathbf{A}\|^2 = 2c'_5 \tilde{\rho}_w^{2k} \|\Pi \mathbf{A}\|^2$$

How to use it?

Define the following matrices:

$$\widetilde{\Pi} = \begin{bmatrix} \Pi & \mathbf{0} \\ \mathbf{0} & \Pi \end{bmatrix}, \quad \mathbf{A}_{\#} = \begin{bmatrix} \mathbf{A} \\ \mathbf{A} \end{bmatrix}, \quad \widetilde{\mathbf{W}} = \begin{bmatrix} (1 + \tilde{\eta}_w) \mathbf{W} & -\tilde{\eta}_w \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}$$

Here, define the parameters:

$$\tilde{\eta}_w = \frac{1}{1 + \sqrt{1 - (1 - \theta)^2}}, \quad \tilde{\rho}_w = \sqrt{\tilde{\eta}_w}, \quad \tilde{\theta}_w = 1 - \tilde{\rho}_w = O(\sqrt{\theta})$$

Then we have the bound:

$$\|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \mathbf{A}_{\#}\|^2 = \|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \widetilde{\Pi} \mathbf{A}_{\#}\|^2 \leq 2c'_5 \tilde{\eta}_w^k \|\Pi \mathbf{A}\|^2 = 2c'_5 \tilde{\rho}_w^{2k} \|\Pi \mathbf{A}\|^2$$

- $\|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \mathbf{A}_{\#}\|^2$ decays R-linearly but not Q-linearly.

How to use it?

Define the following matrices:

$$\widetilde{\Pi} = \begin{bmatrix} \Pi & \mathbf{0} \\ \mathbf{0} & \Pi \end{bmatrix}, \quad \mathbf{A}_{\#} = \begin{bmatrix} \mathbf{A} \\ \mathbf{A} \end{bmatrix}, \quad \widetilde{\mathbf{W}} = \begin{bmatrix} (1 + \tilde{\eta}_w) \mathbf{W} & -\tilde{\eta}_w \mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}$$

Here, define the parameters:

$$\tilde{\eta}_w = \frac{1}{1 + \sqrt{1 - (1 - \theta)^2}}, \quad \tilde{\rho}_w = \sqrt{\tilde{\eta}_w}, \quad \tilde{\theta}_w = 1 - \tilde{\rho}_w = O(\sqrt{\theta})$$

Then we have the bound:

$$\|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \mathbf{A}_{\#}\|^2 = \|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \widetilde{\Pi} \mathbf{A}_{\#}\|^2 \leq 2c'_5 \tilde{\eta}_w^k \|\Pi \mathbf{A}\|^2 = 2c'_5 \tilde{\rho}_w^{2k} \|\Pi \mathbf{A}\|^2$$

- $\|\widetilde{\Pi} \widetilde{\mathbf{W}}^k \mathbf{A}_{\#}\|^2$ decays R-linearly but not Q-linearly.
- Although $\widetilde{\mathbf{W}}$ has sub-spectral radius $\tilde{\rho}_w = 1 - O(\sqrt{\theta})$, it may still hold that $\|\widetilde{\Pi} \widetilde{\mathbf{W}}\|_2 \geq 1$.
- Nonetheless, for the specific structure of $\mathbf{A}_{\#}$, we achieve R-linear convergence.

OGT: Optimal GT

Initial: $\mathbf{y}^0 = \mathbf{z}^0 = \mathbf{u}^0 = \mathbf{q}^0 = \mathbf{x}^0$, $\tilde{\mathbf{z}}^0 = \tilde{\mathbf{u}}^0 = \mathbf{x}_{\#}^0$, $\tilde{\mathbf{g}}^0 = \nabla f(\mathbf{x}^0)_{\#}$

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\tilde{\mathbf{z}}^{k+1} = (1 + \beta)^{-1} \widetilde{\mathbf{W}} \left(\tilde{\mathbf{z}}^k + \beta\mathbf{x}_{\#}^k - \eta(\zeta^k(\nabla f(\mathbf{x}^k)_{\#} - \nabla f(\mathbf{q}^k)_{\#}) + \mathbf{g}_{\#}^k) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha momentum})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k$$

$$\tilde{\mathbf{u}}^{k+1} = \widetilde{\mathbf{W}} \left(\xi^k \mathbf{x}_{\#}^k + (1 - \xi^k) \tilde{\mathbf{u}}^k \right)$$

$$\tilde{\mathbf{g}}^{k+1} = \widetilde{\mathbf{W}} \tilde{\mathbf{g}}^k + \nabla f(\mathbf{q}^{k+1})_{\#} - \nabla f(\mathbf{q}^k)_{\#}$$

$$\mathbf{z}^k = [\tilde{\mathbf{z}}^k]_{1:n}, \quad \mathbf{u}^k = [\tilde{\mathbf{u}}^k]_{1:n}, \quad \mathbf{g}^k = [\tilde{\mathbf{g}}^k]_{1:n}$$

OGT: Optimal GT

Initial: $\mathbf{y}^0 = \mathbf{z}^0 = \mathbf{u}^0 = \mathbf{q}^0 = \mathbf{x}^0$, $\tilde{\mathbf{z}}^0 = \tilde{\mathbf{u}}^0 = \mathbf{x}_{\#}^0$, $\tilde{\mathbf{g}}^0 = \nabla f(\mathbf{x}^0)_{\#}$

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\tilde{\mathbf{z}}^{k+1} = (1 + \beta)^{-1} \widetilde{\mathbf{W}} \left(\tilde{\mathbf{z}}^k + \beta\mathbf{x}_{\#}^k - \eta(\zeta^k(\nabla f(\mathbf{x}^k)_{\#} - \nabla f(\mathbf{q}^k)_{\#}) + \mathbf{g}_{\#}^k) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha momentum})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k$$

$$\tilde{\mathbf{u}}^{k+1} = \widetilde{\mathbf{W}} \left(\xi^k \mathbf{x}_{\#}^k + (1 - \xi^k) \tilde{\mathbf{u}}^k \right)$$

$$\tilde{\mathbf{g}}^{k+1} = \widetilde{\mathbf{W}} \tilde{\mathbf{g}}^k + \nabla f(\mathbf{q}^{k+1})_{\#} - \nabla f(\mathbf{q}^k)_{\#}$$

$$\mathbf{z}^k = [\tilde{\mathbf{z}}^k]_{1:n}, \quad \mathbf{u}^k = [\tilde{\mathbf{u}}^k]_{1:n}, \quad \mathbf{g}^k = [\tilde{\mathbf{g}}^k]_{1:n}$$

- OGT Parameters:

$$\tau = \frac{1}{2}, \quad \alpha, \gamma = O\left(\frac{1}{\sqrt{\kappa}}\right), \quad p, q = O(\tilde{\theta}_w), \quad \eta = O\left(\frac{\tilde{\theta}_w}{\sqrt{\mu L}}\right), \quad \beta = \frac{\mu\eta}{2}$$

OGT: Optimal GT

Initial: $\mathbf{y}^0 = \mathbf{z}^0 = \mathbf{u}^0 = \mathbf{q}^0 = \mathbf{x}^0$, $\tilde{\mathbf{z}}^0 = \tilde{\mathbf{u}}^0 = \mathbf{x}_{\#}^0$, $\tilde{\mathbf{g}}^0 = \nabla f(\mathbf{x}^0)_{\#}$

$$\mathbf{x}^k = (1 - \alpha - \tau)\mathbf{y}^k + \alpha\mathbf{z}^k + \tau\mathbf{u}^k$$

$$\tilde{\mathbf{z}}^{k+1} = (1 + \beta)^{-1} \widetilde{\mathbf{W}} \left(\tilde{\mathbf{z}}^k + \beta\mathbf{x}_{\#}^k - \eta(\zeta^k (\nabla f(\mathbf{x}^k)_{\#} - \nabla f(\mathbf{q}^k)_{\#}) + \mathbf{g}_{\#}^k) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^k + \gamma(\mathbf{z}^{k+1} - \mathbf{z}^k) \quad (\text{similar to Katyusha momentum})$$

$$\mathbf{q}^{k+1} = \xi^k \mathbf{x}^k + (1 - \xi^k) \mathbf{q}^k$$

$$\tilde{\mathbf{u}}^{k+1} = \widetilde{\mathbf{W}} \left(\xi^k \mathbf{x}_{\#}^k + (1 - \xi^k) \tilde{\mathbf{u}}^k \right)$$

$$\tilde{\mathbf{g}}^{k+1} = \widetilde{\mathbf{W}} \tilde{\mathbf{g}}^k + \nabla f(\mathbf{q}^{k+1})_{\#} - \nabla f(\mathbf{q}^k)_{\#}$$

$$\mathbf{z}^k = [\tilde{\mathbf{z}}^k]_{1:n}, \quad \mathbf{u}^k = [\tilde{\mathbf{u}}^k]_{1:n}, \quad \mathbf{g}^k = [\tilde{\mathbf{g}}^k]_{1:n}$$

- OGT Parameters:

$$\tau = \frac{1}{2}, \quad \alpha, \gamma = O\left(\frac{1}{\sqrt{\kappa}}\right), \quad p, q = O(\tilde{\theta}_w), \quad \eta = O\left(\frac{\tilde{\theta}_w}{\sqrt{\mu L}}\right), \quad \beta = \frac{\mu\eta}{2}$$

- Convergence rate:

$$\text{Computation: } O\left(\sqrt{\kappa} \log \frac{1}{\epsilon}\right), \quad \text{Communication: } O\left(\sqrt{\frac{\kappa}{\theta}} \log \frac{1}{\epsilon}\right)$$

- Optimal: matches the lower bounds for decentralized optimization.

APAPC: Accelerated Primal-Dual Algorithm

An equivalent form of NIDS in the PAPC format

$$\mathbf{x}^{k+1/2} = \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}^k) + \mathbf{y}^k \right) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \beta(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1/2}$$

$$\mathbf{x}^{k+1} = \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}^k) + \mathbf{y}^{k+1} \right) \right)$$

APAPC: Accelerated Primal-Dual Algorithm

APAPC (Kovalev et al., 2020) update steps

$$\mathbf{x}_g^k = \tau \mathbf{x}^k + (1 - \tau) \mathbf{x}_f^k$$

$$\mathbf{x}^{k+1/2} = \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}_g^k) + \mathbf{y}^k \right) \right)$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \beta(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1/2}$$

$$\mathbf{x}^{k+1} = \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}_g^k) + \mathbf{y}^{k+1} \right) \right)$$

$$\mathbf{x}_f^{k+1} = \mathbf{x}_g^k + \frac{2\tau}{2 - \tau}(\mathbf{x}^{k+1} - \mathbf{x}^k)$$

APAPC: Accelerated Primal-Dual Algorithm

APAPC (Kovalev et al., 2020) update steps

$$\mathbf{x}_g^k = \tau \mathbf{x}^k + (1 - \tau) \mathbf{x}_f^k$$

$$\mathbf{x}^{k+1/2} = \frac{1}{1 + \eta\alpha} \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}_g^k) + \mathbf{y}^k \right) + \eta\alpha \mathbf{x}_g^k \right)$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \beta(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1/2}$$

$$\mathbf{x}^{k+1} = \frac{1}{1 + \eta\alpha} \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}_g^k) + \mathbf{y}^{k+1} \right) + \eta\alpha \mathbf{x}_g^k \right)$$

$$\mathbf{x}_f^{k+1} = \mathbf{x}_g^k + \frac{2\tau}{2 - \tau} (\mathbf{x}^{k+1} - \mathbf{x}^k)$$

APAPC: Accelerated Primal-Dual Algorithm

APAPC (Kovalev et al., 2020) update steps

$$\begin{aligned}\mathbf{x}_g^k &= \tau \mathbf{x}^k + (1 - \tau) \mathbf{x}_f^k \\ \mathbf{x}^{k+1/2} &= \frac{1}{1 + \eta\alpha} \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}_g^k) + \mathbf{y}^k \right) + \eta\alpha \mathbf{x}_g^k \right) \\ \mathbf{y}^{k+1} &= \mathbf{y}^k + \beta(\mathbf{I} - \mathbf{W})\mathbf{x}^{k+1/2} \\ \mathbf{x}^{k+1} &= \frac{1}{1 + \eta\alpha} \left(\mathbf{x}^k - \eta \left(\nabla f(\mathbf{x}_g^k) + \mathbf{y}^{k+1} \right) + \eta\alpha \mathbf{x}_g^k \right) \\ \mathbf{x}_f^{k+1} &= \mathbf{x}_g^k + \frac{2\tau}{2 - \tau} (\mathbf{x}^{k+1} - \mathbf{x}^k)\end{aligned}$$

- **Parameters:**

$$\alpha = \mu, \quad \tau = \min \left(1, \frac{1}{2} \sqrt{\frac{1 - \lambda_n(\mathbf{W})}{1 - \lambda_2(\mathbf{W})}} \cdot \kappa \right), \quad \eta = \frac{1}{4\tau L}, \quad \beta = \frac{1}{\eta(1 - \lambda_n(\mathbf{W}))}$$

- **Convergence rate:**

$$\text{Computation + Communication: } O \left(\left(\sqrt{\frac{\kappa}{\theta}} + \frac{1}{\theta} \right) \log \frac{1}{\epsilon} \right)$$

OPAPC: Optimized PAPC with Chebyshev Acceleration

Apply Chebyshev acceleration to APAPC to eliminate the dependence on θ .

- **Convergence rate:**

$$\text{Computation: } O\left(\sqrt{\kappa} \log \frac{1}{\epsilon}\right), \quad \text{Communication: } O\left(\sqrt{\frac{\kappa}{\theta}} \log \frac{1}{\epsilon}\right)$$

Outline

Introduction

Average Consensus

Decentralized algorithms

DGD

EXTRA/PG-EXTRA

NIDS

General framework

Accelerated algorithms

OGT: Optimal Gradient Tracking with Loopless Chebyshev Acceleration

OPAPC: Optimized PAPC with Chebyshev Acceleration

Algorithms with Compression

LEAD: Linear convergent decentralized algorithm with compression

Directed Network

APD

Centralized Learning

- **Non-compressed gradient descent:**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \gamma \nabla \mathbf{f}(\mathbf{x}^k) = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^k)$$

- **Compressed gradient framework:**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n \mathbf{g}_i^k$$

where $\mathbf{g}_i^k$ is an approximation of $\nabla f_i(\mathbf{x}^k)$.

Direct Compression (Q-SGD)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k))$$

- Assume all nodes have already converged to the true solution $\mathbf{x}^*$. Then one step of (exact) gradient descent with compression will not, in general, preserve $\mathbf{x}^*$.

Direct Compression (Q-SGD)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k))$$

- Assume all nodes have already converged to the true solution $\mathbf{x}^*$. Then one step of (exact) gradient descent with compression will not, in general, preserve $\mathbf{x}^*$.
- That is,

$$\begin{aligned}\mathbf{x} &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^*)) \\ &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) + \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) - Q(\nabla f_i(\mathbf{x}^*))\end{aligned}$$

- Thus, the only way to keep $\mathbf{x} = \mathbf{x}^*$ is if the compression error averages to zero or $\gamma = 0$.

Direct Compression with Diminishing Stepsize

- Due to randomness in the gradient, SGD typically does not converge with a constant stepsize.
- To ensure convergence, a diminishing stepsize is used—though at the cost of slower convergence compared to full gradient descent.

Direct Compression with Diminishing Stepsize

- Due to randomness in the gradient, SGD typically does not converge with a constant stepsize.
- To ensure convergence, a diminishing stepsize is used—though at the cost of slower convergence compared to full gradient descent.
- With a diminishing stepsize and **unbiased compression**, SGD can still converge, assuming the stochastic gradients are bounded.
- The compression error can be absorbed into the inherent noise of the stochastic gradient.
- The convergence rate and guarantees depend on an upper bound of the stochastic gradient norm.

Error Feedback/Compensation (EF-SGD)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1})$$
$$\mathbf{e}_i^k = \nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1} - Q(\nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1})$$

- **Main idea:** Accumulate the compression error and add it to the next variable to be compressed.

Error Feedback/Compensation (EF-SGD)

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1}) \\ \mathbf{e}_i^k &= \nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1} - Q(\nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1})\end{aligned}$$

- **Main idea:** Accumulate the compression error and add it to the next variable to be compressed.
- Assume all nodes are at the optimal solution $\mathbf{x}^*$. Then:

$$\begin{aligned}\mathbf{x} &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^*) + \mathbf{e}_i) \\ &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) \\ &\quad + \frac{\gamma}{n} \sum_{i=1}^n (\nabla f_i(\mathbf{x}^*) + \mathbf{e}_i - Q(\nabla f_i(\mathbf{x}^*) + \mathbf{e}_i)) - \frac{\gamma}{n} \sum_{i=1}^n \mathbf{e}_i\end{aligned}$$

Error Feedback/Compensation (EF-SGD)

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1}) \\ \mathbf{e}_i^k &= \nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1} - Q(\nabla f_i(\mathbf{x}^k) + \mathbf{e}_i^{k-1})\end{aligned}$$

- **Main idea:** Accumulate the compression error and add it to the next variable to be compressed.
- Assume all nodes are at the optimal solution $\mathbf{x}^*$. Then:

$$\begin{aligned}\mathbf{x} &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^*) + \mathbf{e}_i) \\ &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) \\ &\quad + \frac{\gamma}{n} \sum_{i=1}^n (\nabla f_i(\mathbf{x}^*) + \mathbf{e}_i - Q(\nabla f_i(\mathbf{x}^*) + \mathbf{e}_i)) - \frac{\gamma}{n} \sum_{i=1}^n \mathbf{e}_i\end{aligned}$$

- EF-SGD converges under a diminishing stepsize, assuming the stochastic gradients are bounded.

Difference Compression (DIANA)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i^k + Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k))$$

- **Idea:** Quantize the difference between the gradient and its estimator (Mishchenko et al., 2019; Liu et al., 2020).

Difference Compression (DIANA)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i^k + Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k))$$

- **Idea:** Quantize the difference between the gradient and its estimator (Mishchenko et al., 2019; Liu et al., 2020).
- Assume all nodes are at the optimal solution $\mathbf{x}^*$. Then:

$$\begin{aligned} \mathbf{x} &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i + Q(\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i)) \\ &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) + \frac{\gamma}{n} \sum_{i=1}^n (\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i - Q(\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i)) \end{aligned}$$

Difference Compression (DIANA)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i^k + Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k))$$

- **Idea:** Quantize the difference between the gradient and its estimator (Mishchenko et al., 2019; Liu et al., 2020).
- Assume all nodes are at the optimal solution $\mathbf{x}^*$. Then:

$$\begin{aligned} \mathbf{x} &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i + Q(\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i)) \\ &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) + \frac{\gamma}{n} \sum_{i=1}^n (\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i - Q(\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i)) \end{aligned}$$

- If $\mathbf{h}_i = \nabla f_i(\mathbf{x}^*)$, then the compression error vanishes and $\mathbf{x} = \mathbf{x}^*$.

Difference Compression (DIANA)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i^k + Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k))$$

- **Idea:** Quantize the difference between the gradient and its estimator (Mishchenko et al., 2019; Liu et al., 2020).
- Assume all nodes are at the optimal solution $\mathbf{x}^*$. Then:

$$\begin{aligned} \mathbf{x} &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i + Q(\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i)) \\ &= \mathbf{x}^* - \frac{\gamma}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}^*) + \frac{\gamma}{n} \sum_{i=1}^n (\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i - Q(\nabla f_i(\mathbf{x}^*) - \mathbf{h}_i)) \end{aligned}$$

- If $\mathbf{h}_i = \nabla f_i(\mathbf{x}^*)$, then the compression error vanishes and $\mathbf{x} = \mathbf{x}^*$.
- **Update rule for $\mathbf{h}_i$:**

$$\mathbf{h}_i^{k+1} = \mathbf{h}_i^k + \alpha Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k)$$

Difference Compression (DIANA)

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{\gamma}{n} \sum_{i=1}^n (\mathbf{h}_i^k + Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k))$$

- **Client update:**

$$\mathbf{h}_i^{k+1} = \mathbf{h}_i^k + \alpha Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k)$$

- **Server update (no access to individual $\mathbf{h}_i^k$):**

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \bar{\mathbf{h}}^k - \frac{\gamma}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k)$$

$$\bar{\mathbf{h}}^{k+1} = \bar{\mathbf{h}}^k + \alpha \cdot \frac{1}{n} \sum_{i=1}^n Q(\nabla f_i(\mathbf{x}^k) - \mathbf{h}_i^k)$$

Deterministic Decentralized Algorithm with Compression

Equivalent form of NIDS:

$$\begin{aligned}\mathbf{y}^k &= \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k) - \gamma \mathbf{d}^k \\ \mathbf{d}^{k+1} &= \mathbf{d}^k + \frac{1}{2\gamma} ((\mathbf{I} - \mathbf{W})\mathbf{y}^k) \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k) - \gamma \mathbf{d}^{k+1}\end{aligned}$$

Deterministic Decentralized Algorithm with Compression

Equivalent form of NIDS:

$$\begin{aligned}\mathbf{y}^k &= \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k) - \gamma \mathbf{d}^k \\ \mathbf{d}^{k+1} &= \mathbf{d}^k + \frac{1}{2\gamma} \left((\mathbf{I} - \mathbf{W}) \mathbf{Q}(\mathbf{y}^k - \mathbf{h}^k) + \mathbf{h}^k - \mathbf{W} \mathbf{h}^k \right) \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k) - \gamma \mathbf{d}^{k+1}\end{aligned}$$

- Introduce a local estimator $\mathbf{h}$ for $\mathbf{y}$, and compress the difference $\mathbf{y}^k - \mathbf{h}^k$.
- Update the estimator using a difference compression rule:

$$\mathbf{h}^{k+1} = \mathbf{h}^k + \alpha Q(\mathbf{y}^k - \mathbf{h}^k)$$

Deterministic Decentralized Algorithm with Compression

Equivalent form of NIDS:

$$\begin{aligned}\mathbf{y}^k &= \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k) - \gamma \mathbf{d}^k \\ \mathbf{d}^{k+1} &= \mathbf{d}^k + \frac{1}{2\gamma} \left((\mathbf{I} - \mathbf{W}) \mathbf{Q}(\mathbf{y}^k - \mathbf{h}^k) + \mathbf{h}^k - \bar{\mathbf{y}}^k \right) \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \gamma \nabla f(\mathbf{x}^k) - \gamma \mathbf{d}^{k+1}\end{aligned}$$

- Introduce a local estimator $\mathbf{h}$ for $\mathbf{y}$, and compress the difference $\mathbf{y}^k - \mathbf{h}^k$.
- Update the estimator using a difference compression rule:

$$\mathbf{h}^{k+1} = \mathbf{h}^k + \alpha Q(\mathbf{y}^k - \mathbf{h}^k)$$

- The compressed average is updated as:

$$\bar{\mathbf{y}}^{k+1} = \mathbf{W} \mathbf{h}^{k+1} = \bar{\mathbf{y}}^k + \alpha \mathbf{W} Q(\mathbf{y}^k - \mathbf{h}^k)$$

Convergence Results

- **Constant stepsize:** Let $\gamma \in (0, \frac{2}{\mu+L})$ and assume stochastic gradient noise with variance σ^2 .

$$\mathbb{E}[\mathcal{L}^{k+1}] \leq \rho \mathbb{E}[\mathcal{L}^k] + \gamma^2 \sigma^2$$

where $\mathcal{L}^k$ is a Lyapunov function and $\rho < 1$ indicates geometric decay up to a noise floor.

- **Diminishing stepsize:** Set $\gamma_k = O(1/k)$.

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E} \left\| \mathbf{x}_i^k - \mathbf{x}^* \right\|^2 \lesssim O\left(\frac{1}{k}\right)$$

This yields a sublinear convergence rate in the mean-squared error sense.

Outline

Introduction

Average Consensus

Decentralized algorithms

DGD

EXTRA/PG-EXTRA

NIDS

General framework

Accelerated algorithms

OGT: Optimal Gradient Tracking with Loopless Chebyshev Acceleration

OPAPC: Optimized PAPC with Chebyshev Acceleration

Algorithms with Compression

LEAD: Linear convergent decentralized algorithm with compression

Directed Network

APD

PUSH-DIGing / ADD-OPT

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad x_1 = x_2 = \cdots = x_n$$

under a **column-stochastic matrix** $\mathbf{C}$.

- Column-stochastic: $\mathbf{1}^\top \mathbf{C} = \mathbf{1}^\top$, $\mathbf{C} \geq 0$
- Row-stochastic: $\mathbf{C} \mathbf{1} = \mathbf{1}$, $\mathbf{C} \geq 0$

PUSH-DIGing / ADD-OPT

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad x_1 = x_2 = \cdots = x_n$$

under a **column-stochastic matrix** $\mathbf{C}$.

PUSH-DIGing (Nedić-Olshevsky-Shi, 2017) / **ADD-OPT** (Xi-Xin-Khan, 2017):

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k \quad (\text{auxiliary weights})$$

$$\mathbf{x}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k) \quad (\text{state update})$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f((\mathbf{V}^{k+1})^{-1}\mathbf{x}^{k+1}) - \nabla f((\mathbf{V}^k)^{-1}\mathbf{x}^k) \quad (\text{gradient tracking})$$

- $\mathbf{V}^k = \text{Diag}(\mathbf{v}^k)$, $\mathbf{x}^k = [x_1^{k\top}; \dots; x_n^{k\top}]$
- The gradient tracking update resembles that of PUSH-PULL / AB.

PUSH-DIGing / ADD-OPT

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad x_1 = x_2 = \cdots = x_n$$

under a **column-stochastic matrix** $\mathbf{C}$.

PUSH-DIGing (Nedić-Olshevsky-Shi, 2017) / **ADD-OPT** (Xi-Xin-Khan, 2017):

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k \quad (\text{auxiliary weights})$$

$$\mathbf{x}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k) \quad (\text{state update})$$

$$\mathbf{G}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f(\mathbf{u}^{k+1}) - \nabla f(\mathbf{u}^k) \quad (\text{gradient tracking})$$

- $\mathbf{V}^k = \text{Diag}(\mathbf{v}^k)$, $\mathbf{x}^k = [x_1^{k\top}; \dots; x_n^{k\top}]$
- Define $\mathbf{u}^k = (\mathbf{V}^k)^{-1} \mathbf{x}^k$.

PUSH-DIGing / ADD-OPT

Consider the optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad x_1 = x_2 = \cdots = x_n$$

under a **column-stochastic matrix** $\mathbf{C}$.

PUSH-DIGing (Nedić-Olshevsky-Shi, 2017) / **ADD-OPT** (Xi-Xin-Khan, 2017):

$$\begin{aligned} \mathbf{v}^{k+1} &= \mathbf{C}\mathbf{v}^k && \text{(auxiliary weights)} \\ \mathbf{u}^{k+1} &= (\mathbf{V}^{k+1})^{-1} \mathbf{C}\mathbf{V}^k (\mathbf{u}^k - \gamma(\mathbf{V}^k)^{-1} \mathbf{g}^k) && \text{(normalized update)} \\ \mathbf{G}^{k+1} &= \mathbf{C}\mathbf{g}^k + \nabla f(\mathbf{u}^{k+1}) - \nabla f(\mathbf{u}^k) && \text{(gradient tracking)} \end{aligned}$$

- $\mathbf{V}^k = \text{Diag}(\mathbf{v}^k)$, $\mathbf{x}^k = [x_1^{k\top}; \dots; x_n^{k\top}]$
- Define $\mathbf{u}^k = (\mathbf{V}^k)^{-1} \mathbf{x}^k$.
- $(\mathbf{V}^{k+1})^{-1} \mathbf{C}\mathbf{V}^k$ is row-stochastic.

Averaged PUSH-DIGing / ADD-OPT

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k \quad // \text{ Update local weights (column stochastic)}$$

$$\mathbf{x}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k) \quad // \text{ Local update with local gradient tracking}$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f(\mathbf{u}^{k+1}) - \nabla f(\mathbf{u}^k) \quad // \text{ Gradient tracking at each node}$$

- Assume $\mathbf{g}^0 = \nabla f(\mathbf{u}^0)$ where $\mathbf{u}^k = (\mathbf{V}^k)^{-1} \mathbf{x}^k$. Then for all $k \geq 0$,

$$\bar{\mathbf{g}}^k = \frac{1}{n} \mathbf{1}^\top \nabla f(\mathbf{u}^k).$$

Averaged PUSH-DIGing / ADD-OPT

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k \quad // \text{ Update local weights (column stochastic)}$$

$$\bar{\mathbf{x}}^{k+1} = \bar{\mathbf{x}}^k - \gamma \bar{\mathbf{g}}^k \quad // \text{ Average iterate update}$$

$$\bar{\mathbf{g}}^{k+1} = \bar{\mathbf{g}}^k + \frac{1}{n} \mathbf{1}^\top [\nabla f(\mathbf{u}^{k+1}) - \nabla f(\mathbf{u}^k)] \quad // \text{ Averaged gradient tracking}$$

- Assume $\mathbf{g}^0 = \nabla f(\mathbf{u}^0)$ where $\mathbf{u}^k = (\mathbf{V}^k)^{-1} \mathbf{x}^k$. Then for all $k \geq 0$,

$$\bar{\mathbf{g}}^k = \frac{1}{n} \mathbf{1}^\top \nabla f(\mathbf{u}^k).$$

- Our goal is to control the bias:

$$\bar{\mathbf{g}}^k - \nabla f(\bar{\mathbf{x}}^k).$$

- Even though we lack consensus on $\mathbf{x}_i^k$, we still need $\bar{\mathbf{g}}^k \rightarrow 0$ to guarantee convergence to the minimizer.

AGM (Linear Coupling, Allen-Zhu and Orecchia, 2017)

$$\mathbf{y}^{k+1} = \mathbf{x}^k - \frac{1}{L} \nabla f(\mathbf{x}^k)$$

$$\mathbf{z}^{k+1} = \mathbf{z}^k - \frac{\alpha_k}{L} \nabla f(\mathbf{x}^k)$$

$$\mathbf{x}^{k+1} = (1 - \tau_{k+1})\mathbf{y}^{k+1} + \tau_{k+1}\mathbf{z}^{k+1}$$

- Linear coupling between gradient descent ($\mathbf{y}^k$) and mirror descent ($\mathbf{z}^k$).
- Step sizes: $\alpha_k = O(k)$, $\tau_k = O\left(\frac{1}{k}\right)$.
- Achieves optimal convergence rate $O(1/k^2)$ for smooth convex minimization.

Accelerated Push-DIGing (APD)

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k$$

$$\mathbf{x}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k)$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f((\mathbf{V}^{k+1})^{-1}\mathbf{x}^{k+1}) - \nabla f((\mathbf{V}^k)^{-1}\mathbf{x}^k)$$

Accelerated Push-DIGing (APD)

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k$$

$$\mathbf{y}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k)$$

$$\mathbf{z}^{k+1} = \mathbf{C}(\mathbf{z}^k - \alpha_k \gamma \mathbf{g}^k)$$

$$\mathbf{x}^{k+1} = (1 - \tau_{k+1})\mathbf{y}^{k+1} + \tau_{k+1}\mathbf{z}^{k+1}$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f((\mathbf{V}^{k+1})^{-1}\mathbf{x}^{k+1}) - \nabla f((\mathbf{V}^k)^{-1}\mathbf{x}^k)$$

Accelerated Push-DIGing (APD)

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k$$

$$\mathbf{y}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k)$$

$$\mathbf{z}^{k+1} = \mathbf{C}(\mathbf{z}^k - \alpha_k \gamma \mathbf{g}^k)$$

$$\mathbf{x}^{k+1} = (1 - \tau_{k+1})\mathbf{y}^{k+1} + \tau_{k+1}\mathbf{z}^{k+1}$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f((\mathbf{V}^{k+1})^{-1}\mathbf{x}^{k+1}) - \nabla f((\mathbf{V}^k)^{-1}\mathbf{x}^k)$$

Convergence result (non-strongly convex case):

- Convergence rate: $\mathcal{O}\left(\frac{1}{k^2}\right)$
- Parameter choices:
 - $w_1 \in \left(0, \frac{1}{20}\right], \quad \gamma = \mathcal{O}\left(\frac{1}{L}\right)$
 - $\alpha_k = 1 + w_1 k = \mathcal{O}(k), \quad \tau_k = \frac{1}{4\alpha_k} = \mathcal{O}\left(\frac{1}{k}\right)$

APD-SC: Strongly Convex Case

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k$$

$$\mathbf{y}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k)$$

$$\mathbf{z}^{k+1} = \mathbf{C}((1 - \beta)\mathbf{z}^k + \beta\mathbf{x}^k - \alpha_k\gamma\mathbf{g}^k)$$

$$\mathbf{x}^{k+1} = (1 - \tau_{k+1})\mathbf{y}^{k+1} + \tau_{k+1}\mathbf{z}^{k+1}$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f((\mathbf{V}^{k+1})^{-1}\mathbf{x}^{k+1}) - \nabla f((\mathbf{V}^k)^{-1}\mathbf{x}^k)$$

APD-SC: Strongly Convex Case

$$\mathbf{v}^{k+1} = \mathbf{C}\mathbf{v}^k$$

$$\mathbf{y}^{k+1} = \mathbf{C}(\mathbf{x}^k - \gamma \mathbf{g}^k)$$

$$\mathbf{z}^{k+1} = \mathbf{C}((1 - \beta)\mathbf{z}^k + \beta\mathbf{x}^k - \alpha_k \gamma \mathbf{g}^k)$$

$$\mathbf{x}^{k+1} = (1 - \tau_{k+1})\mathbf{y}^{k+1} + \tau_{k+1}\mathbf{z}^{k+1}$$

$$\mathbf{g}^{k+1} = \mathbf{C}\mathbf{g}^k + \nabla f((\mathbf{V}^{k+1})^{-1}\mathbf{x}^{k+1}) - \nabla f((\mathbf{V}^k)^{-1}\mathbf{x}^k)$$

Convergence result:

- Strongly convex case: $\mathcal{O}\left(\left(1 - c\sqrt{\frac{\mu}{L}}\right)^k\right)$
- Parameter choices:
 - $\eta = \mathcal{O}\left(\frac{1}{L}\right), \quad \beta = \mathcal{O}\left(\sqrt{\frac{\mu}{L}}\right)$
 - $\alpha_k = \mathcal{O}\left(\sqrt{\frac{L}{\mu}}\right), \quad \gamma_k = \frac{1}{4\alpha_k} = \mathcal{O}\left(\sqrt{\frac{\mu}{L}}\right)$

Thank You!

<https://mingyan08.github.io/>

email: yanming@cuhk.edu.cn

My Work in this Area

EXTRA and NIDS:

- Z. Li, W. Shi, and M. Yan, A decentralized proximal-gradient method with network independent step-sizes and separated convergence rates, IEEE TSP, 67 (2019), 4494-4506. (NIDS algorithm)
- H. Tang, X. Lian, M. Yan, C. Zhang and J. Liu, D2: Decentralized training over decentralized data, In: ICML 2018, 4848-4856. (D^2 : stochastic NIDS with nonconvex functions)
- Z. Li and M. Yan, A primal-dual algorithm with optimal stepsizes and its application in decentralized consensus optimization, Advances in Computational Mathematics, 47 (2021), 9. (EXTRA converges with large stepsize $(5 + 3\lambda_{\min})/(8L)$)
- Y. Li and M. Yan, On linear convergence of two decentralized algorithms, Journal of Optimization Theory and Applications, 189 (2021), 271-290. (EXTRA and NIDS converge for averaged strongly convex functions)

Compression

- X. Liu, Y. Li, R. Wang, J. Tang, and M. Yan, Linear convergent decentralized optimization with compression, In: ICLR 2021. (LEAD algorithm)
- Z. Song, L. Shi, S. Pu, and M. Yan, Compressed gradient tracking for decentralized optimization over general directed networks, IEEE TSP, 70(2022), 1775–1787. (Compressed Push-Pull)

Acceleration

- Z. Song, L. Shi, S. Pu, and M. Yan, Optimal gradient tracking for decentralized optimization, Mathematical Programming, 207(2024), 1–53. (OGT algorithm)
- Z. Song, L. Shi, S. Pu, and M. Yan, Provably accelerated decentralized gradient method over unbalanced directed graphs, SIAM Journal on Optimization, 34(2024), 1131–1156. (APD: accelerated Push-DIGing)

Others

- Z. Song, W. Li, K. Jin, L. Shi, M. Yan, W. Yin, K. Yuan, Communication-efficient topologies for decentralized learning with $O(1)$ consensus rate, In: NeurIPS 2022. (changing 1-peer Network)
- S. Alghunaim, Q. Lyu, M. Yan, and A. Sayed, Dual consensus proximal algorithm for multi-agent sharing problems, IEEE TSP, 69(2021), 5568–5579. (Agents connected under constraints)